

Slip 1

Q.1) Write a C Menu driven Program to implement following functionality a) Accept Available b) Display Allocation, Max c) Display the contents of need matrix d) Display Available Process Allocation Max Available A B C A B C P0 2 3 2 9 7 5 3 3 2 P1 4 0 0 5 2 2 P2 5 0 4 1 0 4 P3 4 3 3 4 4 4 P4 2 2 4 6 5 5

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int allocation[5][3] = {
```

```
        {2, 3, 2},
```

```
        {4, 0, 0},
```

```
        {5, 0, 4},
```

```
        {4, 3, 3},
```

```
        {2, 2, 4}
```

```
    };
```

```
    int max[5][3] = {
```

```
        {9, 7, 5},
```

```
        {5, 2, 2},
```

```
        {1, 0, 4},
```

```
        {4, 4, 4},
```

```
        {6, 0, 5}
```

```
    };
```

```
    int need[5][3];
```

```
    int available[3];
```

```
    int choice, i, j;
```

```
    /* Calculate Need Matrix */
```

```
    for(i = 0; i < 5; i++)
```

```
    {
```

```
        for(j = 0; j < 3; j++)
```

```
        {
```

```
            need[i][j] = max[i][j] - allocation[i][j];
```

```
        }
```

```
}
```

```
do
```

```
{
```

```
    printf("\n\n----- BANKER'S ALGORITHM -----");
```

```
    printf("\n1. Accept Available");
```

```
    printf("\n2. Display Allocation and Max");
```

```
    printf("\n3. Display Need Matrix");
```

```
    printf("\n4. Display Available");
```

```
    printf("\n5. Exit");
```

```
    printf("\nEnter your choice: ");
```

```
    scanf("%d", &choice);
```

```
    switch(choice)
```

```
    {
```

```
        case 1:
```

```
            printf("\nEnter Available resources A B C: ");
```

```
            scanf("%d %d %d",
```

```
                &available[0],
```

```
                &available[1],
```

```
                &available[2]);
```

```
            printf("Available resources accepted.\n");
```

```
            break;
```

```
        case 2:
```

```
            printf("\nProcess\tAllocation\tMax");
```

```
            printf("\n\tA B C\t\tA B C");
```

```
            for(i = 0; i < 5; i++)
```

```
            {
```

```
                printf("\nP%d\t", i);
```

```
    for(j = 0; j < 3; j++)
        printf("%d ", allocation[i][j]);

    printf("\t\t");

    for(j = 0; j < 3; j++)
        printf("%d ", max[i][j]);
}
break;
```

case 3:

```
    printf("\nNeed Matrix:");
    printf("\nProcess\tA B C");
```

```
    for(i = 0; i < 5; i++)
    {
        printf("\nP%d\t", i);

        for(j = 0; j < 3; j++)
            printf("%d ", need[i][j]);
    }
    break;
```

case 4:

```
    printf("\nAvailable Resources:");
    printf("\nA\tB\tC\n");
```

```
    for(j = 0; j < 3; j++)
        printf("%d\t", available[j]);
```

```
    break;
```

case 5:

```

        printf("\nProgram Ended.");
        break;

    default:
        printf("\nInvalid choice!");
    }

} while(choice != 5);

return 0;
}

```

.2 Write a C program that behaves like a shell which displays the command prompt '\$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'count' as a. \$ count c filename: It will display the number of characters in given file b. \$ count w filename: It will display the number of words in given file c. \$ count l filename: It will display the number of lines in given file [

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<unistd.h>
#include<sys/wait.h>

int main(){
    char command[100],*args[10];
    int i;
    while(1){
        printf("$ ");
        fgets(command,100,stdin);
        command[strlen(command)-1]='\0';
        i=0;
        args[i]=strtok(command," ");
        while(args[i]!=NULL){
            i++;

```

```

args[i]=strtok(NULL, " ");
}
if(strcmp(args[0],"exit")==0) break;
if(strcmp(args[0],"count")==0){
FILE *fp;
char ch;
int count=0;
fp=fopen(args[2],"r");
if(fp==NULL){
printf("File not found\n");
continue;
}
if(args[1][0]=='c'){
while((ch=fgetc(fp))!=EOF) count++;
printf("Characters=%d\n",count);
}
else if(args[1][0]=='w'){
int inword=0;
while((ch=fgetc(fp))!=EOF){
if(ch==' '||ch=='\n'||ch=='\t') inword=0;
else if(inword==0){
count++;
inword=1;
}
}
printf("Words=%d\n",count);
}
else if(args[1][0]=='l'){
while((ch=fgetc(fp))!=EOF)
if(ch=='\n') count++;
printf("Lines=%d\n",count);
}
}

```

```

fclose(fp);
}
else{
if(fork()==0){
execvp(args[0],args);
printf("Command not found\n");
exit(0);
}
else wait(NULL);
}
}
return 0;
}

```

Slip 2

Write a program to simulate Linked file allocation method. Assume disk with 'n' number of blocks. Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option. ☐ Show Bit Vector ☐ Create New File ☐ Show Directory ☐ Exit

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<time.h>
```

```

struct file{
char name[20];
int start;
int blocks[20];
int n;
}f[10];

```

```

int main(){
int n,bit[50],freeb[50],i,j,k=0,choice,count=0;

```

```
printf("Enter number of blocks: ");  
scanf("%d",&n);
```

```
srand(time(0));  
for(i=0;i<n;i++)  
bit[i]=rand()%2;
```

```
for(i=0;i<n;i++)  
if(bit[i]==0)  
freeb[k++]=i;
```

```
while(1){  
printf("\n1.Show Bit Vector");  
printf("\n2.Create New File");  
printf("\n3.Show Directory");  
printf("\n4.Exit");  
printf("\nEnter choice: ");  
scanf("%d",&choice);
```

```
if(choice==1){  
printf("\nBit Vector:\n");  
for(i=0;i<n;i++)  
printf("%d ",bit[i]);  
}
```

```
else if(choice==2){  
printf("\nEnter file name: ");  
scanf("%s",f[count].name);  
printf("Enter number of blocks: ");  
scanf("%d",&f[count].n);
```

```
if(f[count].n>k){
```

```
printf("Not enough free blocks!");  
continue;  
}
```

```
f[count].start=freeb[0];
```

```
for(i=0;i<f[count].n;i++){  
f[count].blocks[i]=freeb[i];  
bit[freeb[i]]=1;  
}
```

```
k=0;  
for(i=0;i<n;i++){  
if(bit[i]==0)  
freeb[k++]=i;
```

```
printf("File created successfully.");  
count++;  
}
```

```
else if(choice==3){  
printf("\nFile\tStart\tBlocks");  
for(i=0;i<count;i++){  
printf("\n%s\t%d\t",f[i].name,f[i].start);  
for(j=0;j<f[i].n;j++){  
printf("%d ",f[i].blocks[j]);  
}  
}
```

```
else if(choice==4)  
break;
```

```

else
printf("Invalid choice!");
}
return 0;
}

```

Q2.. Write a simulation program for disk scheduling using FCFS algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments. 55, 58, 39, 18, 90, 160, 150, 38, 184 Start Head Position: 50

```

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, m, head, i, total = 0, r[20];

    printf("Enter total number of disk blocks: ");
    scanf("%d", &n);

    printf("Enter number of requests: ");
    scanf("%d", &m);

    printf("Enter request string:\n");
    for(i = 0; i < m; i++)
        scanf("%d", &r[i]);

    printf("Enter current head position: ");
    scanf("%d", &head);

    printf("\nOrder of service: %d", head);

    for(i = 0; i < m; i++)
    {

```

```

        total += abs(r[i] - head);

        head = r[i];

        printf(" -> %d", head);
    }

    printf("\nTotal head movement = %d\n", total);

    return 0;
}

```

Slip 3

Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance. Consider the following snapshot of system, A, B, C and D is the resource type. Process Allocation Max Available A B C D A B C D A B C D P0 0 0 1 2 0 0 1 2 1 5 2 0 P1 1 0 0 0 1 7 5 0 P2 1 3 5 4 2 3 5 6 P3 0 6 3 2 0 6 5 2 P4 0 0 1 4 0 6 5 6 a) Calculate and display the content of need matrix? b) Is the system in safe state? If display the safe sequence.

```
#include <stdio.h>
```

```

int main()
{
    int alloc[5][4] = {
        {0,0,1,2},
        {1,0,0,0},
        {1,3,5,4},
        {0,6,3,2},
        {0,0,1,4}
    };

```

```

    int max[5][4] = {
        {0,0,1,2},
        {1,7,5,0},
        {2,3,5,6},

```

```
    {0,6,5,2},  
    {0,6,5,6}  
};
```

```
int avail[4] = {1,5,2,0};  
int need[5][4], finish[5] = {0};  
int safe[5], count = 0, i, j, k, found;
```

```
/* Calculate Need */
```

```
for(i = 0; i < 5; i++)  
    for(j = 0; j < 4; j++)  
        need[i][j] = max[i][j] - alloc[i][j];
```

```
printf("Need Matrix:\n");  
printf("  A B C D\n");
```

```
for(i = 0; i < 5; i++)  
    printf("P%d : %d %d %d %d\n", i,  
        need[i][0], need[i][1],  
        need[i][2], need[i][3]);
```

```
/* Banker's Algorithm */
```

```
while(count < 5)  
{  
    found = 0;  
  
    for(i = 0; i < 5; i++)  
    {  
        if(finish[i] == 0)  
        {  
            for(j = 0; j < 4; j++)  
                if(need[i][j] > avail[j])
```

```

        break;

    if(j == 4)
    {
        for(k = 0; k < 4; k++)
            avail[k] += alloc[i][k];

        safe[count++] = i;
        finish[i] = 1;
        found = 1;
    }
}

if(found == 0)
    break;
}

if(count == 5)
{
    printf("\nSystem is in SAFE state.\n");
    printf("Safe Sequence: ");

    for(i = 0; i < 5; i++)
    {
        printf("P%d", safe[i]);

        if(i < 4)
            printf(" -> ");
    }
    printf("\n");
}
else

```

```
printf("\nSystem is NOT in safe state.\n");
```

```
return 0;
```

```
}
```

Q.2 Write a simulation program for disk scheduling using SSTF algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments. 98, 183, 37, 122, 14, 124, 65 Start Head Position: 53

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
int n, head, i, j, min, pos, total = 0;
```

```
int req[20], done[20];
```

```
printf("Enter total number of disk blocks: ");
```

```
scanf("%d", &n);
```

```
printf("Enter number of requests: ");
```

```
scanf("%d", &n);
```

```
printf("Enter request string:\n");
```

```
for(i = 0; i < n; i++)
```

```
scanf("%d", &req[i]);
```

```
printf("Enter current head position: ");
```

```
scanf("%d", &head);
```

```
for(i = 0; i < n; i++)
```

```
done[i] = 0;
```

```

printf("\nOrder of service: %d", head);

for(i = 0; i < n; i++)
{
    min = 9999;

    for(j = 0; j < n; j++)
    {
        if(done[j] == 0 && abs(req[j] - head) < min)
        {
            min = abs(req[j] - head);
            pos = j;
        }
    }

    total += min;
    head = req[pos];
    done[pos] = 1;

    printf(" -> %d", head);
}

printf("\nTotal head movement = %d\n", total);

return 0;
}

```

Slip 4

Q.1 Implement the Menu driven Banker's algorithm for accepting Allocation, Max from user a) Accept Available b) Display Allocation, Max c) Find Need and display It, d) Display Available Consider the system with 3 resources types A, B, and C with 7, 2, 6 instances respectively consider the following snapshot: Process Allocation Request A B C A B C P0 0 1 0 0 0 0 P1 4 0 0 5 2 2 P2 5 0 4 1 0 4 P3 4 3 3 4 4 4 P4 2 2 4 6 5 5

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int allocation[5][3] = {
```

```
        {0, 1, 0},
```

```
        {4, 0, 0},
```

```
        {5, 0, 4},
```

```
        {4, 3, 3},
```

```
        {2, 2, 4}
```

```
    };
```

```
    int max[5][3] = {
```

```
        {0, 0, 0},
```

```
        {5, 2, 2},
```

```
        {1, 0, 4},
```

```
        {4, 4, 4},
```

```
        {6, 5, 5}
```

```
    };
```

```
    int need[5][3];
```

```
    int available[3];
```

```
    int choice, i, j;
```

```
    /* Calculate Need Matrix */
```

```
    for(i = 0; i < 5; i++)
```

```
    {
```

```
        for(j = 0; j < 3; j++)
```

```
        {
```

```
            need[i][j] = max[i][j] - allocation[i][j];
```

```
        }
```

```
}
```

```
do
```

```
{
```

```
    printf("\n\n----- BANKER'S ALGORITHM -----");
```

```
    printf("\n1. Accept Available");
```

```
    printf("\n2. Display Allocation and Max");
```

```
    printf("\n3. Display Need Matrix");
```

```
    printf("\n4. Display Available");
```

```
    printf("\n5. Exit");
```

```
    printf("\nEnter your choice: ");
```

```
    scanf("%d", &choice);
```

```
    switch(choice)
```

```
    {
```

```
        case 1:
```

```
            printf("\nEnter Available resources A B C: ");
```

```
            scanf("%d %d %d",
```

```
                &available[0],
```

```
                &available[1],
```

```
                &available[2]);
```

```
            printf("Available resources accepted.\n");
```

```
            break;
```

```
        case 2:
```

```
            printf("\nProcess\tAllocation\tMax");
```

```
            printf("\n\tA B C\t\tA B C");
```

```
            for(i = 0; i < 5; i++)
```

```
            {
```

```
printf("\nP%d\t", i);
```

```
for(j = 0; j < 3; j++)
```

```
    printf("%d ", allocation[i][j]);
```

```
printf("\t\t");
```

```
for(j = 0; j < 3; j++)
```

```
    printf("%d ", max[i][j]);
```

```
}
```

```
break;
```

case 3:

```
printf("\nNeed Matrix:");
```

```
printf("\nProcess\tA B C");
```

```
for(i = 0; i < 5; i++)
```

```
{
```

```
    printf("\nP%d\t", i);
```

```
for(j = 0; j < 3; j++)
```

```
    printf("%d ", need[i][j]);
```

```
}
```

```
break;
```

case 4:

```
printf("\nAvailable Resources:");
```

```
printf("\nA\tB\tC\n");
```

```
for(j = 0; j < 3; j++)
```

```
    printf("%d\t", available[j]);
```

```

        break;

    case 5:
        printf("\nProgram Ended.");
        break;

    default:
        printf("\nInvalid choice!");
    }

} while(choice != 5);

return 0;
}

```

Write a simulation program for disk scheduling using SCAN algorithm. Accept total Number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.
82, 170, 43, 140, 24, 16, 190, 65 Starting Head position= 50 Direction: Left

```

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int blocks, n, head, i, j, temp, total = 0;
    int req[20];

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);

    printf("Enter number of requests: ");

```

```
scanf("%d", &n);
```

```
printf("Enter request string:\n");
```

```
for(i = 0; i < n; i++)
```

```
    scanf("%d", &req[i]);
```

```
printf("Enter current head position: ");
```

```
scanf("%d", &head);
```

```
/* Sort requests */
```

```
for(i = 0; i < n - 1; i++)
```

```
{
```

```
    for(j = i + 1; j < n; j++)
```

```
    {
```

```
        if(req[i] > req[j])
```

```
        {
```

```
            temp = req[i];
```

```
            req[i] = req[j];
```

```
            req[j] = temp;
```

```
        }
```

```
    }
```

```
}
```

```
printf("\nOrder of service: %d", head);
```

```
/* Move Left */
```

```
for(i = n - 1; i >= 0; i--)
```

```
{
```

```
    if(req[i] < head)
```

```
    {
```

```
        total += abs(req[i] - head);
```

```
        head = req[i];
```

```

        printf(" -> %d", head);
    }
}

/* Move to 0 */
total += head;
head = 0;
printf(" -> %d", head);

/* Move Right */
for(i = 0; i < n; i++)
{
    if(req[i] > head)
    {
        total += abs(req[i] - head);
        head = req[i];
        printf(" -> %d", head);
    }
}

printf("\nTotal head movement = %d\n", total);

return 0;
}

```

Write a simulation program for disk scheduling using SCAN algorithm. Accept total Number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments.

82, 170, 43, 140, 24, 16, 190, 65 Starting Head position= 50 Direction: Left

```

#include <stdio.h>
#include <stdlib.h>

```

```

int main()
{

```

```
int blocks, n, head, i, j, temp, total = 0;
```

```
int req[20];
```

```
printf("Enter total number of disk blocks: ");
```

```
scanf("%d", &blocks);
```

```
printf("Enter number of requests: ");
```

```
scanf("%d", &n);
```

```
printf("Enter request string:\n");
```

```
for(i = 0; i < n; i++)
```

```
    scanf("%d", &req[i]);
```

```
printf("Enter current head position: ");
```

```
scanf("%d", &head);
```

```
/* Sort requests */
```

```
for(i = 0; i < n - 1; i++)
```

```
{
```

```
    for(j = i + 1; j < n; j++)
```

```
    {
```

```
        if(req[i] > req[j])
```

```
        {
```

```
            temp = req[i];
```

```
            req[i] = req[j];
```

```
            req[j] = temp;
```

```
        }
```

```
    }
```

```
}
```

```
printf("\nOrder of service: %d", head);
```

```

/* Move Left */
for(i = n - 1; i >= 0; i--)
{
    if(req[i] < head)
    {
        total += abs(req[i] - head);
        head = req[i];
        printf(" -> %d", head);
    }
}

/* Move to 0 */
total += head;
head = 0;
printf(" -> %d", head);

/* Move Right */
for(i = 0; i < n; i++)
{
    if(req[i] > head)
    {
        total += abs(req[i] - head);
        head = req[i];
        printf(" -> %d", head);
    }
}

printf("\nTotal head movement = %d\n", total);

return 0;
}

```

Slip 5

1 Write the simulation program for demand paging and show the page scheduling total and number of page faults according the FIFO page replacement algorithm. Assume the memory of n frames. Reference String: 3,4,5,6,3,4,7,3,4,5,6,7,2,4,6

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i, j, page, pos = 0;
```

```
    int pages[20] = {3,4,5,6,3,4,7,3,4,5,6,7,2,4,6};
```

```
    int frame[10];
```

```
    int faults = 0, hit;
```

```
    printf("Enter number of frames: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++)
```

```
        frame[i] = -1;
```

```
    printf("\nPage\tFrames\t\tStatus\n");
```

```
    for(i = 0; i < 15; i++)
```

```
    {
```

```
        page = pages[i];
```

```
        hit = 0;
```

```
        for(j = 0; j < n; j++)
```

```
        {
```

```
            if(frame[j] == page)
```

```
            {
```

```
                hit = 1;
```

```
                break;
```

```
            }
```

```
        }
```

```

if(hit == 0)
{
    frame[pos] = page;
    pos = (pos + 1) % n;
    faults++;
}

printf("%d\t", page);

for(j = 0; j < n; j++)
{
    if(frame[j] == -1)
        printf("- ");
    else
        printf("%d ", frame[j]);
}

if(hit == 0)
    printf("\tPage Fault");
else
    printf("\tHit");

printf("\n");
}

printf("\nTotal Page Faults = %d\n", faults);
printf("Total Page Hits = %d\n", 15 - faults);

return 0;
}

```

2 Consider a system with 'n' processes and 'm' resource types. Accept number of instances for every resource type. For each process accept the allocation and maximum

requirement matrices. Write a program to display the contents of need matrix and to check if the given request of a process can be granted immediately or not. (Use resource request algorithm)

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, m, i, j, p;
```

```
    int allocation[10][10], max[10][10], need[10][10];
```

```
    int total[10], available[10], request[10];
```

```
    int possible = 1;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resource types: ");
```

```
    scanf("%d", &m);
```

```
    /* Accept total instances */
```

```
    printf("Enter total instances of each resource:\n");
```

```
    for(i = 0; i < m; i++)
```

```
        scanf("%d", &total[i]);
```

```
    /* Accept Allocation */
```

```
    printf("\nEnter Allocation Matrix:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        printf("P%d: ", i);
```

```
        for(j = 0; j < m; j++)
```

```
            scanf("%d", &allocation[i][j]);
```

```
    }
```

```
    /* Accept Maximum */
```

```
printf("\nEnter Maximum Matrix:\n");
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    printf("P%d: ", i);
```

```
    for(j = 0; j < m; j++)
```

```
        scanf("%d", &max[i][j]);
```

```
}
```

```
/* Calculate Available */
```

```
for(j = 0; j < m; j++)
```

```
{
```

```
    available[j] = total[j];
```

```
    for(i = 0; i < n; i++)
```

```
        available[j] -= allocation[i][j];
```

```
}
```

```
/* Calculate Need */
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    for(j = 0; j < m; j++)
```

```
        need[i][j] = max[i][j] - allocation[i][j];
```

```
}
```

```
/* Display Need */
```

```
printf("\nNeed Matrix:\n");
```

```
printf("Process\t");
```

```
for(j = 0; j < m; j++)
```

```
    printf("R%d ", j);
```

```
for(i = 0; i < n; i++)
```

```

{
    printf("\nP%d\t", i);

    for(j = 0; j < m; j++)
        printf("%d ", need[i][j]);
}

/* Display Available */
printf("\n\nAvailable Resources:\n");
for(j = 0; j < m; j++)
    printf("%d ", available[j]);

/* Accept Request */
printf("\n\nEnter process number for request: ");
scanf("%d", &p);

printf("Enter request of P%d:\n", p);
for(j = 0; j < m; j++)
    scanf("%d", &request[j]);

/* Check Request <= Need */
for(j = 0; j < m; j++)
{
    if(request[j] > need[p][j])
    {
        possible = 0;
        break;
    }
}

/* Check Request <= Available */
if(possible == 1)

```

```

{
    for(j = 0; j < m; j++)
    {
        if(request[j] > available[j])
        {
            possible = 0;
            break;
        }
    }
}

if(possible == 1)
    printf("\nRequest can be GRANTED immediately.\n");
else
    printf("\nRequest CANNOT be granted immediately.\n");

return 0;
}

```

Slip 6

Write a program to simulate Sequential file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driver program with menu options as mentioned below and implement each option. ☐ Show Bit Vector ☐ Create New File ☐ Show Directory ☐ Exit

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int n, i, choice, start, length, j, free;
```

```
    int bit[100], startBlock[20], lengthFile[20], files = 0;
```

```

printf("Enter number of disk blocks: ");
scanf("%d", &n);

/* Randomly allocate some blocks */
for(i = 0; i < n; i++)
    bit[i] = rand() % 2;

do
{
    printf("\n\n----- SEQUENTIAL FILE ALLOCATION -----");
    printf("\n1. Show Bit Vector");
    printf("\n2. Create New File");
    printf("\n3. Show Directory");
    printf("\n4. Exit");

    printf("\nEnter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            printf("\nBit Vector:\n");

            for(i = 0; i < n; i++)
                printf("%d ", bit[i]);

            break;

        case 2:
            printf("\nEnter starting block: ");
            scanf("%d", &start);

```

```
printf("Enter number of blocks: ");  
scanf("%d", &length);
```

```
free = 1;
```

```
/* Check free blocks */
```

```
for(i = start; i < start + length; i++)  
{  
    if(i >= n || bit[i] == 1)  
    {  
        free = 0;  
        break;  
    }  
}
```

```
if(free == 1)  
{  
    for(i = start; i < start + length; i++)  
        bit[i] = 1;
```

```
    startBlock[files] = start;  
    lengthFile[files] = length;  
    files++;
```

```
    printf("File created successfully.");  
}
```

```
else
```

```
    printf("Blocks are not free.");
```

```
break;
```

```
case 3:
```

```

printf("\nDirectory:\n");
printf("File\tStart\tLength\n");

for(i = 0; i < files; i++)
    printf("F%d\t%d\t%d\n",
        i + 1, startBlock[i], lengthFile[i]);

break;

case 4:
    printf("\nProgram Ended.");
    break;

default:
    printf("\nInvalid choice!");
}

} while(choice != 4);

return 0;
}

```

Write a simulation program for disk scheduling using C-SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. also display the total number of head movements (Assume disk size=200) 15, 45, 75, 105, 135, 165, 195, 80 Starting Head Position: 100 Direction: Right

```

#include <stdio.h>
#include <stdlib.h>

```

```

int main()
{
    int blocks, n, head, i, j, temp, total = 0;
    int req[20];

```

```
printf("Enter total number of disk blocks: ");  
scanf("%d", &blocks);
```

```
printf("Enter number of requests: ");  
scanf("%d", &n);
```

```
printf("Enter request string:\n");  
for(i = 0; i < n; i++)  
    scanf("%d", &req[i]);
```

```
printf("Enter current head position: ");  
scanf("%d", &head);
```

```
/* Sort requests */  
for(i = 0; i < n - 1; i++)  
{  
    for(j = i + 1; j < n; j++)  
    {  
        if(req[i] > req[j])  
        {  
            temp = req[i];  
            req[i] = req[j];  
            req[j] = temp;  
        }  
    }  
}
```

```
printf("\nOrder of service: %d", head);
```

```
/* Move Right */  
for(i = 0; i < n; i++)  
{
```

```

    if(req[i] >= head)
    {
        total += abs(req[i] - head);
        head = req[i];
        printf(" -> %d", head);
    }
}

/* Move to last block */
total += (blocks - 1) - head;
head = blocks - 1;
printf(" -> %d", head);

/* Jump to first block */
total += head;
head = 0;
printf(" -> %d", head);

/* Serve remaining requests */
for(i = 0; i < n; i++)
{
    if(req[i] < 100)
    {
        total += abs(req[i] - head);
        head = req[i];
        printf(" -> %d", head);
    }
}

printf("\nTotal head movement = %d\n", total);

return 0;

```

}

Slip 7

Q.1 Consider the following snapshot of the system. Process Allocation Max Available A B C
D A B C D A B C D P0 2 0 0 1 4 2 1 2 3 3 2 1 P1 3 1 2 1 5 2 5 2 P2 2 1 0 3 2 3 1 6 P3 1
3 1 2 1 4 2 4 P4 1 4 3 2 3 6 6 5 Using Resource –Request algorithm to Check
whether the current system is in safe state or not

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int alloc[5][4] = {
```

```
        {2,0,0,1},
```

```
        {3,1,2,1},
```

```
        {2,1,0,3},
```

```
        {1,3,1,2},
```

```
        {1,4,3,2}
```

```
    };
```

```
    int max[5][4] = {
```

```
        {4,2,1,2},
```

```
        {5,2,5,2},
```

```
        {2,3,1,6},
```

```
        {1,4,2,4},
```

```
        {3,6,6,5}
```

```
    };
```

```
    int avail[4] = {3,3,2,1};
```

```
    int need[5][4], finish[5] = {0};
```

```
    int safe[5], count = 0;
```

```
    int i, j, k, found;
```

```
    /* Calculate Need */
```

```
    for(i = 0; i < 5; i++)
```

```

for(j = 0; j < 4; j++)
    need[i][j] = max[i][j] - alloc[i][j];

printf("Need Matrix:\n");
printf("  A B C D\n");

for(i = 0; i < 5; i++)
    printf("P%d : %d %d %d %d\n", i,
        need[i][0], need[i][1],
        need[i][2], need[i][3]);

/* Resource Request / Banker's Algorithm */
while(count < 5)
{
    found = 0;

    for(i = 0; i < 5; i++)
    {
        if(finish[i] == 0)
        {
            for(j = 0; j < 4; j++)
            {
                if(need[i][j] > avail[j])
                    break;
            }

            if(j == 4)
            {
                for(k = 0; k < 4; k++)
                    avail[k] += alloc[i][k];

                safe[count++] = i;
            }
        }
    }
}

```

```
        finish[i] = 1;
        found = 1;
    }
}

if(found == 0)
    break;
}

if(count == 5)
{
    printf("\nSystem is in SAFE state.\n");
    printf("Safe Sequence: ");

    for(i = 0; i < 5; i++)
    {
        printf("P%d", safe[i]);

        if(i < 4)
            printf(" -> ");
    }

    printf("\n");
}
else
{
    printf("\nSystem is NOT in SAFE state.\n");
}

return 0;
}
```

. Write a program that demonstrates the use of nice() system call. After a process is started using fork(), assign higher priority to the child using nice() system call.

child

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    int pid;
```

```
    pid = fork();
```

```
    if(pid < 0)
```

```
    {
```

```
        printf("Fork failed\n");
```

```
    }
```

```
    else if(pid == 0)
```

```
    {
```

```
        printf("Child process started\n");
```

```
        nice(-5);
```

```
        printf("Child process priority changed\n");
```

```
        printf("Child PID = %d\n", getpid());
```

```
    }
```

```
    else
```

```
    {
```

```
        wait(NULL);
```

```
        printf("Parent process\n");
```

```
        printf("Parent PID = %d\n", getpid());
```

```
    }
```

```
    return 0;
}
```

Slip 9

Write the simulation program for demand paging and show the page scheduling and total number of page faults according the optimal page replacement algorithm. Assume the memory of 'n' frames. Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

```
#include <stdio.h>
```

```
int main()
{
    int n, i, j, k, page, pos;
    int pages[20] = {8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
    int frame[10];
    int faults = 0, hit, farthest;

    printf("Enter number of frames: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
        frame[i] = -1;

    printf("\nPage\tFrames\t\tStatus\n");

    for(i = 0; i < 15; i++)
    {
        page = pages[i];
        hit = 0;

        for(j = 0; j < n; j++)
        {
```

```
if(frame[j] == page)
{
    hit = 1;
    break;
}
}
```

```
if(hit == 0)
{
    faults++;
```

```
/* Find empty frame */
```

```
for(j = 0; j < n; j++)
```

```
{
    if(frame[j] == -1)
    {
        frame[j] = page;
        break;
    }
}
```

```
/* Optimal replacement */
```

```
if(j == n)
```

```
{
    farthest = -1;
    pos = -1;
```

```
for(j = 0; j < n; j++)
```

```
{
    for(k = i + 1; k < 15; k++)
    {
        if(frame[j] == pages[k])
```

```

        break;
    }

    if(k > farthest)
    {
        farthest = k;
        pos = j;
    }
}

frame[pos] = page;
}
}

printf("%d\t", page);

for(j = 0; j < n; j++)
{
    if(frame[j] == -1)
        printf("- ");
    else
        printf("%d ", frame[j]);
}

if(hit == 0)
    printf("\tPage Fault");
else
    printf("\tHit");

printf("\n");
}

```

```

printf("\nTotal Page Faults = %d\n", faults);
printf("Total Page Hits = %d\n", 15 - faults);

return 0;
}

```

Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. the arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int n, i, currentTime = 0, completed = 0;
    int at[20], bt[20], ct[20], wt[20], tat[20];
    int done[20] = {0};
    int min, pos;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("\nEnter Arrival Time and First CPU Burst:\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d Arrival Time: ", i + 1);
        scanf("%d", &at[i]);
    }
}

```

```
    printf("P%d First CPU Burst: ", i + 1);  
    scanf("%d", &bt[i]);  
}
```

```
srand(time(0));
```

```
for(i = 0; i < n; i++)  
    bt[i] = bt[i] + rand() % 5;
```

```
printf("\nGenerated CPU Burst:\n");
```

```
for(i = 0; i < n; i++)  
    printf("P%d = %d\n", i + 1, bt[i]);
```

```
printf("\nGantt Chart:\n");
```

```
while(completed < n)  
{  
    min = 999;  
    pos = -1;  
  
    for(i = 0; i < n; i++)  
    {  
        if(done[i] == 0 && at[i] <= currentTime && bt[i] < min)  
        {  
            min = bt[i];  
            pos = i;  
        }  
    }  
}
```

```
if(pos == -1)  
{
```

```

        currentTime++;
        continue;
    }

    printf("| P%d ", pos + 1);

    currentTime = currentTime + bt[pos];

    ct[pos] = currentTime;
    tat[pos] = ct[pos] - at[pos];
    wt[pos] = tat[pos] - bt[pos];

    done[pos] = 1;
    completed++;
}

printf("\n");

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

float avgWT = 0, avgTAT = 0;

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i],
        tat[i], wt[i]);

    avgWT += wt[i];
    avgTAT += tat[i];
}

```

```

printf("\nAverage Waiting Time = %.2f", avgWT / n);
printf("\nAverage Turnaround Time = %.2f\n", avgTAT / n);

return 0;
}

```

Slip 10

Write the program to simulate Round Robin (RR) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int n, i, currentTime = 0, completed = 0;
    int tq;
    int at[20], bt[20], rem[20], ct[20], wt[20], tat[20];
    int queue[100], front = 0, rear = 0;
    int visited[20] = {0};

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("\nEnter Arrival Time and First CPU Burst:\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d Arrival Time: ", i + 1);
        scanf("%d", &at[i]);
    }

```

```

    printf("P%d First CPU Burst: ", i + 1);
    scanf("%d", &bt[i]);
}

printf("\nEnter Time Quantum: ");
scanf("%d", &tq);

srand(time(0));

/* Generate next CPU burst randomly */
for(i = 0; i < n; i++)
    bt[i] = bt[i] + rand() % 5;

for(i = 0; i < n; i++)
    rem[i] = bt[i];

printf("\nGenerated CPU Burst:\n");

for(i = 0; i < n; i++)
    printf("P%d = %d\n", i + 1, bt[i]);

/* Add processes which have already arrived */
for(i = 0; i < n; i++)
{
    if(at[i] == 0)
    {
        queue[rear++] = i;
        visited[i] = 1;
    }
}

printf("\nGantt Chart:\n");

```

```

while(completed < n)
{
    /* If queue is empty, move time forward */
    if(front == rear)
    {
        currentTime++;

        for(i = 0; i < n; i++)
        {
            if(visited[i] == 0 && at[i] <= currentTime)
            {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }

        continue;
    }

    i = queue[front++];

    printf("| P%d ", i + 1);

    if(rem[i] > tq)
    {
        currentTime = currentTime + tq;
        rem[i] = rem[i] - tq;
    }
    else
    {
        currentTime = currentTime + rem[i];
    }
}

```

```

    rem[i] = 0;

    ct[i] = currentTime;
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    completed++;
}

/* Add newly arrived processes */
int j;
for(j = 0; j < n; j++)
{
    if(visited[j] == 0 && at[j] <= currentTime)
    {
        queue[rear++] = j;
        visited[j] = 1;
    }
}

/* Put unfinished process back into queue */
if(rem[i] > 0)
    queue[rear++] = i;
}

printf("\n");

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

float avgWT = 0, avgTAT = 0;

for(i = 0; i < n; i++)

```

```

{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i],
        tat[i], wt[i]);

    avgWT = avgWT + wt[i];
    avgTAT = avgTAT + tat[i];
}

printf("\nAverage Waiting Time = %.2f", avgWT / n);
printf("\nAverage Turnaround Time = %.2f\n", avgTAT / n);

return 0;
}

```

Write a simulation program for disk scheduling using Look algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of request in the order in which it is served. Also display the total number of head moments.
 55, 58, 39, 18, 90, 160, 150, 38. Start Head Position: 100 Direction: Left

```

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int blocks, n, head, i, j, temp, total = 0;
    int req[20];

    printf("Enter total number of disk blocks: ");
    scanf("%d", &blocks);

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request string:\n");

```

```

for(i = 0; i < n; i++)
    scanf("%d", &req[i]);

printf("Enter current head position: ");
scanf("%d", &head);

/* Sort requests */
for(i = 0; i < n - 1; i++)
{
    for(j = i + 1; j < n; j++)
    {
        if(req[i] > req[j])
        {
            temp = req[i];
            req[i] = req[j];
            req[j] = temp;
        }
    }
}

printf("\nOrder of service: %d", head);

/* Move Right */
for(i = 0; i < n; i++)
{
    if(req[i] >= head)
    {
        total += abs(req[i] - head);
        head = req[i];
        printf(" -> %d", head);
    }
}

```

```

/* Reverse and move Left */
for(i = n - 1; i >= 0; i--)
{
    if(req[i] < head)
    {
        total += abs(req[i] - head);
        head = req[i];
        printf(" -> %d", head);
    }
}

printf("\nTotal head movement = %d\n", total);

return 0;
}

```

Slip 11

----- Q.1 Write a C program to simulate Banker's algorithm for the purpose of Deadlock avoidance. The following snapshot of system, A, B, C and D are the resource type. Process Allocation Max Available A B C A B C A B C P0 0 1 0 0 0 0 0 0 0 P1 2 0 0 2 0 2 P2 3 0 3 0 0 0 P3 2 1 1 1 0 0 P4 0 0 2 0 0 2 Implement the following Menu. a. Accept Available b. Display Allocation, Max c. Display the contents of need matrix d. Display Available

```
#include <stdio.h>
```

```

int main()
{
    int alloc[5][3] = {
        {0,1,0},
        {2,0,0},
        {3,0,3},
        {2,1,1},
        {0,0,2}
    }
}

```

```
};
```

```
int max[5][3] = {  
    {0,0,0},  
    {2,0,2},  
    {3,0,3},  
    {2,1,1},  
    {0,0,2}  
};
```

```
int avail[3];  
int need[5][3];  
int i, j, choice;
```

```
for(i = 0; i < 5; i++)  
    for(j = 0; j < 3; j++)  
        need[i][j] = max[i][j] - alloc[i][j];
```

```
while(1)  
{  
    printf("\n--- BANKER'S ALGORITHM ---\n");  
    printf("1. Accept Available\n");  
    printf("2. Display Allocation, Max\n");  
    printf("3. Display Need Matrix\n");  
    printf("4. Display Available\n");  
    printf("5. Exit\n");  
    printf("Enter choice: ");  
    scanf("%d", &choice);  
  
    switch(choice)  
    {  
        case 1:
```

```
printf("\nEnter Available resources A B C: ");
scanf("%d %d %d",
      &avail[0], &avail[1], &avail[2]);
printf("Available resources accepted.\n");
break;
```

case 2:

```
printf("\nAllocation Matrix:\n");
printf("A B C\n");
```

```
for(i = 0; i < 5; i++)
{
    printf("P%d ", i);
    for(j = 0; j < 3; j++)
        printf("%d ", alloc[i][j]);
    printf("\n");
}
```

```
printf("\nMax Matrix:\n");
printf("A B C\n");
```

```
for(i = 0; i < 5; i++)
{
    printf("P%d ", i);
    for(j = 0; j < 3; j++)
        printf("%d ", max[i][j]);
    printf("\n");
}
break;
```

case 3:

```
printf("\nNeed Matrix:\n");
```

```
printf("A B C\n");
```

```
for(i = 0; i < 5; i++)
```

```
{
```

```
    printf("P%d ", i);
```

```
    for(j = 0; j < 3; j++)
```

```
        printf("%d ", need[i][j]);
```

```
    printf("\n");
```

```
}
```

```
break;
```

```
case 4:
```

```
    printf("\nAvailable Resources:\n");
```

```
    printf("A B C\n");
```

```
    printf("%d %d %d\n",
```

```
        avail[0], avail[1], avail[2]);
```

```
    break;
```

```
case 5:
```

```
    return 0;
```

```
default:
```

```
    printf("Invalid choice!\n");
```

```
}
```

```
}
```

```
return 0;
```

```
}
```

Write the simulation program for demand paging and show the page scheduling and total number of page faults according the optimal page replacement algorithm. Assume the memory of n frames. Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i, j, k, page, pos;
```

```
    int pages[20] = {8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
```

```
    int frame[10];
```

```
    int faults = 0, hit, farthest;
```

```
    printf("Enter number of frames: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++)
```

```
        frame[i] = -1;
```

```
    printf("\nPage\tFrames\t\tStatus\n");
```

```
    for(i = 0; i < 15; i++)
```

```
    {
```

```
        page = pages[i];
```

```
        hit = 0;
```

```
        for(j = 0; j < n; j++)
```

```
        {
```

```
            if(frame[j] == page)
```

```
            {
```

```
                hit = 1;
```

```
                break;
```

```
            }
```

```
        }
```

```
        if(hit == 0)
```

```

{
    faults++;

    /* Find empty frame */
    for(j = 0; j < n; j++)
    {
        if(frame[j] == -1)
        {
            frame[j] = page;
            break;
        }
    }

    /* Optimal replacement */
    if(j == n)
    {
        farthest = -1;
        pos = -1;

        for(j = 0; j < n; j++)
        {
            for(k = i + 1; k < 15; k++)
            {
                if(frame[j] == pages[k])
                    break;
            }

            if(k > farthest)
            {
                farthest = k;
                pos = j;
            }
        }
    }
}

```

```

    }

    frame[pos] = page;
}
}

printf("%d\t", page);

for(j = 0; j < n; j++)
{
    if(frame[j] == -1)
        printf("- ");
    else
        printf("%d ", frame[j]);
}

if(hit == 0)
    printf("\tPage Fault");
else
    printf("\tHit");

printf("\n");
}

printf("\nTotal Page Faults = %d\n", faults);
printf("Total Page Hits = %d\n", 15 - faults);

return 0;
}

```

Slip 12

Write the program to simulate FCFS CPU scheduling. The arrival time and first CPU- burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
int main()
```

```
{
```

```
    int n, i, currentTime = 0;
```

```
    int at[20], bt[20], ct[20], wt[20], tat[20];
```

```
    float avgWT = 0, avgTAT = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("\nEnter Arrival Time and First CPU Burst:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        printf("P%d Arrival Time: ", i + 1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("P%d First CPU Burst: ", i + 1);
```

```
        scanf("%d", &bt[i]);
```

```
    }
```

```
    srand(time(0));
```

```
    /* Generate next CPU burst randomly */
```

```
    for(i = 0; i < n; i++)
```

```
        bt[i] = bt[i] + rand() % 5;
```

```
printf("\nGenerated CPU Burst:\n");
```

```
for(i = 0; i < n; i++)
```

```
    printf("P%d = %d\n", i + 1, bt[i]);
```

```
printf("\nGantt Chart:\n");
```

```
/* FCFS Scheduling */
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    /* If CPU is idle */
```

```
    if(currentTime < at[i])
```

```
        currentTime = at[i];
```

```
    printf("| P%d ", i + 1);
```

```
    currentTime = currentTime + bt[i];
```

```
    ct[i] = currentTime;
```

```
    tat[i] = ct[i] - at[i];
```

```
    wt[i] = tat[i] - bt[i];
```

```
}
```

```
printf("\n");
```

```
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
```

```
        i + 1, at[i], bt[i], ct[i],
```

```

        tat[i], wt[i]);

    avgWT = avgWT + wt[i];
    avgTAT = avgTAT + tat[i];
}

printf("\nAverage Waiting Time = %.2f", avgWT / n);
printf("\nAverage Turnaround Time = %.2f\n", avgTAT / n);

return 0;
}

```

2 Implement the C program to accept n integers to be sorted. Main function creates child process using fork system call. Parent process sorts the integers using bubble sort and waits for child process using wait system call child process sorts the integers using inse

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int n, a[20], i, j, temp;
    pid_t pid;

    printf("Enter number of integers: ");
    scanf("%d", &n);

    printf("Enter integers:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    pid = fork();

    if(pid < 0)

```

```

{
    printf("Fork failed\n");
    return 1;
}

/* Child process - Insertion Sort */
if(pid == 0)
{
    printf("\nChild Process - Insertion Sort\n");

    for(i = 1; i < n; i++)
    {
        temp = a[i];
        j = i - 1;

        while(j >= 0 && a[j] > temp)
        {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = temp;
    }

    printf("Sorted array: ");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
}

/* Parent process - Bubble Sort */

```

```

else
{
    printf("\nParent Process - Bubble Sort\n");

    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Sorted array: ");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");

    wait(NULL);

    printf("Parent waited for child process.\n");
}

return 0;
}

```

Slip 13

Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
int main()
```

```
{
```

```
    int n, i, j, currentTime = 0;
```

```
    int at[20], bt[20], ct[20], wt[20], tat[20];
```

```
    int done[20] = {0};
```

```
    int min, pos, temp;
```

```
    float avgWT = 0, avgTAT = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("\nEnter Arrival Time and First CPU Burst:\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        printf("P%d Arrival Time: ", i + 1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("P%d First CPU Burst: ", i + 1);
```

```
        scanf("%d", &bt[i]);
```

```
    }
```

```
    srand(time(0));
```

```
    /* Generate next CPU burst randomly */
```

```
    for(i = 0; i < n; i++)
```

```
bt[i] = bt[i] + rand() % 5;
```

```
printf("\nGenerated CPU Burst:\n");
```

```
for(i = 0; i < n; i++)
```

```
    printf("P%d = %d\n", i + 1, bt[i]);
```

```
printf("\nGantt Chart:\n");
```

```
/* Non-Preemptive SJF */
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    min = 999;
```

```
    pos = -1;
```

```
/* Find shortest arrived process */
```

```
for(j = 0; j < n; j++)
```

```
{
```

```
    if(done[j] == 0 && at[j] <= currentTime)
```

```
    {
```

```
        if(bt[j] < min)
```

```
        {
```

```
            min = bt[j];
```

```
            pos = j;
```

```
        }
```

```
    }
```

```
}
```

```
/* If no process has arrived */
```

```
if(pos == -1)
```

```
{
```

```
    currentTime++;
```

```

        i--;
        continue;
    }

    printf("| P%d ", pos + 1);

    currentTime = currentTime + bt[pos];

    ct[pos] = currentTime;
    tat[pos] = ct[pos] - at[pos];
    wt[pos] = tat[pos] - bt[pos];

    done[pos] = 1;
}

printf("\n");

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i],
        tat[i], wt[i]);

    avgWT = avgWT + wt[i];
    avgTAT = avgTAT + tat[i];
}

printf("\nAverage Waiting Time = %.2f", avgWT / n);
printf("\nAverage Turnaround Time = %.2f\n", avgTAT / n);

```

```
    return 0;
}
```

Write a simulation program for disk scheduling using LOOK algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head movements
15, 30, 55, 90, 125, 140, 170 Starting Head Position: 100 Direction: Left

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int blocks, n, head, i, j, temp, total = 0;
```

```
    int req[20];
```

```
    printf("Enter total number of disk blocks: ");
```

```
    scanf("%d", &blocks);
```

```
    printf("Enter number of requests: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter request string:\n");
```

```
    for(i = 0; i < n; i++)
```

```
        scanf("%d", &req[i]);
```

```
    printf("Enter current head position: ");
```

```
    scanf("%d", &head);
```

```
    /* Sort requests */
```

```
    for(i = 0; i < n - 1; i++)
```

```
    {
```

```
        for(j = i + 1; j < n; j++)
```

```
        {
```

```

        if(req[i] > req[j])
        {
            temp = req[i];
            req[i] = req[j];
            req[j] = temp;
        }
    }
}

```

```

printf("\nOrder of service: %d", head);

```

```

/* Move Left */
for(i = n - 1; i >= 0; i--)
{
    if(req[i] <= head)
    {
        total += abs(req[i] - head);
        head = req[i];
        printf(" -> %d", head);
    }
}

```

```

/* Reverse and move Right */
for(i = 0; i < n; i++)
{
    if(req[i] > head)
    {
        total += abs(req[i] - head);
        head = req[i];
        printf(" -> %d", head);
    }
}

```

```

printf("\nTotal head movement = %d\n", total);

return 0;
}

```

Slip 14

1. Implement the C program that accepts an integer array. Main function forks child process. Parent process sorts an integer array and passes the sorted array to child process through the command line arguments of `execve()` system call. The child process uses `execve()` system call to load new program that uses this sorted array for performing the binary search the particular item in the array.

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int n, i, j, temp, item;
    int a[20];
    char *args[25];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter item to search: ");
    scanf("%d", &item);

```

```
/* Bubble Sort */
```

```
for(i = 0; i < n - 1; i++)
```

```
{
```

```
    for(j = 0; j < n - i - 1; j++)
```

```
    {
```

```
        if(a[j] > a[j + 1])
```

```
        {
```

```
            temp = a[j];
```

```
            a[j] = a[j + 1];
```

```
            a[j + 1] = temp;
```

```
        }
```

```
    }
```

```
}
```

```
printf("\nSorted Array: ");
```

```
for(i = 0; i < n; i++)
```

```
    printf("%d ", a[i]);
```

```
printf("\n");
```

```
pid_t pid = fork();
```

```
if(pid == 0)
```

```
{
```

```
    char nstr[10], itemstr[10];
```

```
    sprintf(nstr, "%d", n);
```

```
    sprintf(itemstr, "%d", item);
```

```
    args[0] = "./child";
```

```
    args[1] = nstr;
```

```

    for(i = 0; i < n; i++)
    {
        char *str = malloc(10);
        sprintf(str, "%d", a[i]);
        args[i + 2] = str;
    }

    args[n + 2] = itemstr;
    args[n + 3] = NULL;

    execve("./child", args, NULL);

    printf("execve failed\n");
}
else
{
    wait(NULL);
    printf("Parent process completed.\n");
}

return 0;
}

```

Write the simulation program for demand paging and show the page scheduling and total number of page faults according the FIFO page replacement algorithm. Assume the memory of n frames. Reference String: 3, 4, 5, 6, 3, 4, 7, 3, 4, 5, 6, 7, 2, 4, 6

```
#include <stdio.h>
```

```

int main()
{
    int pages[] = {3,4,5,6,3,4,7,3,4,5,6,7,2,4,6};
    int n = 15;
    int frame[10], frames;

```

```

int i, j, k, found, pos = 0;
int faults = 0, hits = 0;

printf("Enter number of frames: ");
scanf("%d", &frames);

for(i = 0; i < frames; i++)
    frame[i] = -1;

printf("\nPage\tFrames\t\tStatus\n");

for(i = 0; i < n; i++)
{
    found = 0;

    /* Check if page is already present */
    for(j = 0; j < frames; j++)
    {
        if(frame[j] == pages[i])
        {
            found = 1;
            break;
        }
    }

    /* Page Fault */
    if(found == 0)
    {
        frame[pos] = pages[i];
        pos = (pos + 1) % frames;
        faults++;
    }
}

```

```

        else
        {
            hits++;
        }

    printf("%d\t", pages[i]);

    for(k = 0; k < frames; k++)
    {
        if(frame[k] == -1)
            printf("- ");
        else
            printf("%d ", frame[k]);
    }

    if(found == 0)
        printf("\tPage Fault");
    else
        printf("\tHit");

    printf("\n");
}

printf("\nTotal Page Faults = %d", faults);
printf("\nTotal Hits = %d\n", hits);

return 0;
}

```

Slip 15

Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child has finished its task. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid)

```
#include <stdio.h>

#include <unistd.h>

#include <stdlib.h>

int main()
{
    int pid;

    pid = fork();

    if(pid < 0)
    {
        printf("Fork failed\n");
    }
    else if(pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID = %d\n", getpid());
        printf("Parent PID = %d\n", getppid());

        sleep(5);

        printf("\nAfter parent terminates:\n");
        printf("Child PID = %d\n", getpid());
        printf("New Parent PID = %d\n", getppid());
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID = %d\n", getpid());

        sleep(2);
```

```

        printf("Parent process terminated\n");
        exit(0);
    }

    return 0;
}

```

Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time .

```

#include <stdio.h>

#include <stdlib.h>

#include <time.h>

int main()
{
    int n, i, time = 0, completed = 0;
    int at[20], bt[20], rem[20], priority[20];
    int ct[20], tat[20], wt[20];
    int min, p, first = 1;
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("\nEnter Arrival Time for P%d: ", i + 1);
        scanf("%d", &at[i]);

        printf("Enter First CPU Burst for P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }
}

```

```

printf("Enter Priority for P%d: ", i + 1);
scanf("%d", &priority[i]);

rem[i] = bt[i];
}

/* Generate next CPU burst randomly */
srand(time(0));

for(i = 0; i < n; i++)
{
    bt[i] = bt[i] + rand() % 5;
    rem[i] = bt[i];
}

printf("\nGenerated CPU Bursts:\n");
for(i = 0; i < n; i++)
    printf("P%d = %d\n", i + 1, bt[i]);

printf("\nGantt Chart:\n");

while(completed < n)
{
    min = 999;
    p = -1;

    /* Find highest priority process */
    for(i = 0; i < n; i++)
    {
        if(at[i] <= time && rem[i] > 0)
        {

```

```

        if(priority[i] < min)
        {
            min = priority[i];
            p = i;
        }
    }
}

```

```

/* No process has arrived */
if(p == -1)
{
    time++;
    continue;
}

```

```

if(first)
{
    printf("| P%d ", p + 1);
    first = 0;
}
else
{
    printf("| P%d ", p + 1);
}

```

```

/* Execute for 1 unit - preemptive */
rem[p]--;
time++;

```

```

if(rem[p] == 0)
{
    ct[p] = time;
}

```

```

        completed++;
    }
}

printf("\n");

/* Calculate TAT and WT */
for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    avgwt = avgwt + wt[i];
    avgtat = avgtat + tat[i];
}

printf("\nProcess\tAT\tBT\tPriority\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], priority[i],
        ct[i], tat[i], wt[i]);
}

printf("\nAverage Waiting Time = %.2f", avgwt / n);
printf("\nAverage Turnaround Time = %.2f\n", avgtat / n);

return 0;
}

```

Slip 16

Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames. Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int pages[] = {3,5,7,2,5,1,2,3,1,3,5,3,1,6,2};
```

```
    int frame[10], counter[10];
```

```
    int n = 15, f;
```

```
    int i, j, k, pos, min, found;
```

```
    int faults = 0;
```

```
    printf("Enter number of frames: ");
```

```
    scanf("%d", &f);
```

```
    for(i = 0; i < f; i++)
```

```
    {
```

```
        frame[i] = -1;
```

```
        counter[i] = 0;
```

```
    }
```

```
    printf("\nPage\tFrames\tStatus\n");
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        found = 0;
```

```
        /* Check page */
```

```
        for(j = 0; j < f; j++)
```

```
        {
```

```
            if(frame[j] == pages[i])
```

```
{  
    found = 1;  
    counter[j] = i;  
    break;  
}  
}
```

```
/* Page Fault */
```

```
if(found == 0)
```

```
{  
    pos = -1;
```

```
/* Find empty frame */
```

```
for(j = 0; j < f; j++)
```

```
{  
    if(frame[j] == -1)  
    {  
        pos = j;  
        break;  
    }
```

```
}
```

```
/* Find LRU page */
```

```
if(pos == -1)
```

```
{  
    min = counter[0];  
    pos = 0;
```

```
for(j = 1; j < f; j++)
```

```
{  
    if(counter[j] < min)  
    {
```

```

        min = counter[j];
        pos = j;
    }
}

frame[pos] = pages[i];
counter[pos] = i;
faults++;
}

printf("%d\t", pages[i]);

for(k = 0; k < f; k++)
{
    if(frame[k] == -1)
        printf("- ");
    else
        printf("%d ", frame[k]);
}

if(found == 0)
    printf("\tFault");
else
    printf("\tHit");

printf("\n");
}

printf("\nTotal Page Faults = %d\n", faults);

return 0;

```

```
}
```

Write a C program that behaves like a shell which displays the command [15 Marks]
prompt '\$'. It accepts the command, tokenize the command line and execute it by
creating the child process. Also implement the additional command 'count' as a \$ count c
filename: It will display the number of characters in given file b \$ count w filename: It
will display the number of words in given file c \$ count l filename: It will display the
number of lines in given file

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<unistd.h>
#include<sys/wait.h>
int main(){
char command[100],*args[10];
int i;
while(1){
printf("$ ");
fgets(command,100,stdin);
command[strlen(command)-1]='\0';
i=0;
args[i]=strtok(command," ");
while(args[i]!=NULL){
i++;
args[i]=strtok(NULL," ");
}
if(strcmp(args[0],"exit")==0) break;
if(strcmp(args[0],"count")==0){
FILE *fp;
char ch;
int count=0;
fp=fopen(args[2],"r");
if(fp==NULL){
printf("File not found\n");
continue;
```

```

}
if(args[1][0]=='c'){
while((ch=fgetc(fp))!=EOF) count++;
printf("Characters=%d\n",count);
}
else if(args[1][0]=='w'){
int inword=0;
while((ch=fgetc(fp))!=EOF){
if(ch==' '||ch=='\n'||ch=='\t') inword=0;
else if(inword==0){
count++;
inword=1;
}
}
printf("Words=%d\n",count);
}
else if(args[1][0]=='l'){
while((ch=fgetc(fp))!=EOF)
if(ch=='\n') count++;
printf("Lines=%d\n",count);
}
fclose(fp);
}
else{
if(fork()==0){
execvp(args[0],args);
printf("Command not found\n");
exit(0);
}
else wait(NULL);
}
}
}

```

```
return 0;
}
```

Slip 17

Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

int main()
{
    int n,i,currentTime=0,count=0,min,p;
    int at[20],bt[20],wt[20],tat[20],done[20]={0};
    float awt=0,atat=0;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        printf("Enter AT and first BT for P%d: ",i+1);
        scanf("%d%d",&at[i],&bt[i]);
    }

    srand(time(0));

    /* Generate next CPU burst randomly */
    for(i=0;i<n;i++)
```

```
bt[i] = bt[i] + rand()%5;
```

```
printf("\nGantt Chart:\n");
```

```
while(count<n)
```

```
{
```

```
    min=9999;
```

```
    p=-1;
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        if(done[i]==0 && at[i]<=currentTime && bt[i]<min)
```

```
        {
```

```
            min=bt[i];
```

```
            p=i;
```

```
        }
```

```
    }
```

```
    if(p== -1)
```

```
    {
```

```
        currentTime++;
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("| P%d ",p+1);
```

```
        currentTime = currentTime + bt[p];
```

```
        tat[p] = currentTime - at[p];
```

```
        wt[p] = tat[p] - bt[p];
```

```
        done[p]=1;
```

```

        count++;
    }
}

printf("\n");

printf("\nProcess\tAT\tBT\tWT\tTAT\n");

for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\n",
        i+1,at[i],bt[i],wt[i],tat[i]);

    awt = awt + wt[i];
    atat = atat + tat[i];
}

printf("\nAverage Waiting Time = %.2f",awt/n);
printf("\nAverage Turnaround Time = %.2f\n",atat/n);

return 0;
}

```

Write a program to simulate Sequential (Contiguous) file allocation method. Assume disk with n number of blocks. Give value of n as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks. Write menu driven program with menu options as mentioned below and implement each option. a. Show Bit Vector b. Create New File c. Show Directory d. Delete File e. Exit

```

#include <stdio.h>
#include <stdlib.h>

```

```

int main()
{
    int n, i, choice, start, length, j, free;

```

```

int bit[100];
int startBlock[20], lengthFile[20];
int files = 0, fileNo;

printf("Enter number of disk blocks: ");
scanf("%d", &n);

/* Randomly allocate some blocks */
for(i = 0; i < n; i++)
    bit[i] = rand() % 2;

do
{
    printf("\n\n--- SEQUENTIAL FILE ALLOCATION ---");
    printf("\n1. Show Bit Vector");
    printf("\n2. Create New File");
    printf("\n3. Show Directory");
    printf("\n4. Delete File");
    printf("\n5. Exit");

    printf("\nEnter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            printf("\nBit Vector:\n");

            for(i = 0; i < n; i++)
                printf("%d ", bit[i]);

            break;

```

case 2:

```
printf("\nEnter starting block: ");
```

```
scanf("%d", &start);
```

```
printf("Enter number of blocks: ");
```

```
scanf("%d", &length);
```

```
free = 1;
```

```
/* Check free blocks */
```

```
for(i = start; i < start + length; i++)
```

```
{
```

```
    if(i >= n || bit[i] == 1)
```

```
    {
```

```
        free = 0;
```

```
        break;
```

```
    }
```

```
}
```

```
if(free == 1)
```

```
{
```

```
    for(i = start; i < start + length; i++)
```

```
        bit[i] = 1;
```

```
    startBlock[files] = start;
```

```
    lengthFile[files] = length;
```

```
    files++;
```

```
    printf("File created successfully.");
```

```
}
```

```
else
```

```
{  
    printf("Blocks are not free.");  
}
```

```
break;
```

case 3:

```
printf("\nDirectory:\n");  
printf("File\tStart\tLength\n");
```

```
for(i = 0; i < files; i++)  
{  
    printf("F%d\t%d\t%d\n",  
        i + 1,  
        startBlock[i],  
        lengthFile[i]);  
}
```

```
break;
```

case 4:

```
printf("\nEnter file number to delete: ");  
scanf("%d", &fileNo);
```

```
if(fileNo >= 1 && fileNo <= files)  
{  
    start = startBlock[fileNo - 1];  
    length = lengthFile[fileNo - 1];
```

```
/* Free the blocks */
```

```
for(i = start; i < start + length; i++)  
    bit[i] = 0;
```

```

        /* Shift files */
        for(j = fileNo - 1; j < files - 1; j++)
        {
            startBlock[j] = startBlock[j + 1];
            lengthFile[j] = lengthFile[j + 1];
        }

        files--;

        printf("File deleted successfully.");
    }
    else
    {
        printf("Invalid file number.");
    }

    break;

case 5:
    printf("\nProgram Ended.");
    break;

default:
    printf("\nInvalid choice!");
}

} while(choice != 5);

return 0;
}

```

Slip 18

Write the program to simulate Preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU burst for different n number of processes should be input to the algorithm. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

int main()
{
    int n, i, currentTime = 0, completed = 0;
    int at[20], bt[20], rem[20], wt[20], tat[20];
    int min, p, finish[20];
    float awt = 0, atat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter AT and first BT for P%d: ", i + 1);
        scanf("%d%d", &at[i], &bt[i]);
    }

    /* Generate next CPU burst randomly */
    srand(time(0));

    for(i = 0; i < n; i++)
    {
        bt[i] = bt[i] + rand() % 5;
        rem[i] = bt[i];
    }
```

```
printf("\nGantt Chart:\n");
```

```
while(completed < n)
```

```
{
```

```
    min = 9999;
```

```
    p = -1;
```

```
    /* Find process with smallest remaining time */
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        if(at[i] <= currentTime && rem[i] > 0 &&
```

```
            rem[i] < min)
```

```
        {
```

```
            min = rem[i];
```

```
            p = i;
```

```
        }
```

```
    }
```

```
    if(p == -1)
```

```
    {
```

```
        currentTime++;
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("| P%d ", p + 1);
```

```
        rem[p]--;
```

```
        currentTime++;
```

```
        /* Process completed */
```

```
        if(rem[p] == 0)
```

```
        {
```

```

        finish[p] = currentTime;
        completed++;
    }
}
}

printf("\n");

/* Calculate WT and TAT */
for(i = 0; i < n; i++)
{
    tat[i] = finish[i] - at[i];
    wt[i] = tat[i] - bt[i];

    awt = awt + wt[i];
    atat = atat + tat[i];
}

printf("\nProcess\tAT\tBT\tWT\tTAT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time = %.2f", awt / n);
printf("\nAverage Turnaround Time = %.2f\n", atat / n);

return 0;
}
#include <stdio.h>

```

```
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int pid;

    pid = fork();

    if(pid < 0)
    {
        printf("Fork failed");
    }
    else if(pid == 0)
    {
        printf("Child process started\n");
        printf("Child PID = %d\n", getpid());
        printf("Parent PID = %d\n", getppid());

        sleep(5);

        printf("\nAfter parent terminates:\n");
        printf("Child PID = %d\n", getpid());
        printf("New Parent PID = %d\n", getppid());
    }
    else
    {
        printf("Parent process\n");
        printf("Parent PID = %d\n", getpid());

        sleep(2);
```

```

        printf("Parent terminated\n");
        exit(0);
    }

    return 0;
}

```

Slip 19

Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. The next CPU-burst should be generated.

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int n, i, currentTime = 0, completed = 0;
    int at[20], bt[20], rem[20], pr[20];
    int finish[20], wt[20], tat[20];
    int p, min;
    float awt = 0, atat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter AT, BT and Priority for P%d: ", i + 1);
        scanf("%d%d%d", &at[i], &bt[i], &pr[i]);
    }

    /* Generate next CPU burst randomly */

```

```

srand(time(0));

for(i = 0; i < n; i++)
{
    bt[i] = bt[i] + rand() % 5;
    rem[i] = bt[i];
}

printf("\nGantt Chart:\n");

while(completed < n)
{
    min = 9999;
    p = -1;

    /* Find highest priority process */
    for(i = 0; i < n; i++)
    {
        if(at[i] <= currentTime && rem[i] > 0 &&
            pr[i] < min)
        {
            min = pr[i];
            p = i;
        }
    }

    if(p == -1)
    {
        currentTime++;
    }
    else
    {

```

```

printf("| P%d ", p + 1);

rem[p]--;
currentTime++;

if(rem[p] == 0)
{
    finish[p] = currentTime;
    completed++;
}
}
}

printf("\n");

/* Calculate WT and TAT */
for(i = 0; i < n; i++)
{
    tat[i] = finish[i] - at[i];
    wt[i] = tat[i] - bt[i];

    awt = awt + wt[i];
    atat = atat + tat[i];
}

printf("\nProcess\tAT\tBT\tPriority\tWT\tTAT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], pr[i],
        wt[i], tat[i]);
}

```

```
}
```

```
printf("\nAverage Waiting Time = %.2f", awt / n);
```

```
printf("\nAverage Turnaround Time = %.2f\n", atat / n);
```

```
return 0;
```

```
}
```

Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int pid;
```

```
    pid = fork();
```

```
    if(pid < 0)
```

```
    {
```

```
        printf("Fork failed");
```

```
    }
```

```
    else if(pid == 0)
```

```
    {
```

```
        printf("Child process started\n");
```

```
        printf("Child PID = %d\n", getpid());
```

```
        printf("Parent PID = %d\n", getppid());
```

```
        sleep(5);
```

```
        printf("\nAfter parent terminates:\n");
```

```
        printf("Child PID = %d\n", getpid());
```

```

        printf("New Parent PID = %d\n", getppid());
    }
    else
    {
        printf("Parent process\n");
        printf("Parent PID = %d\n", getpid());

        sleep(2);

        printf("Parent terminated\n");
        exit(0);
    }

    return 0;
}

```

Slip 20

1 Write a simulation program for disk scheduling using LOOK algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments. 86, 147, 91, 177, 45, 12, 130 Starting Head Position: 60 Direction: Right

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int n, i, j, head, total = 0;
```

```
    int req[100], temp;
```

```
    printf("Enter total number of disk blocks: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of requests: ");
```

```

int r;

scanf("%d", &r);

printf("Enter request string:\n");

for(i = 0; i < r; i++)
    scanf("%d", &req[i]);

printf("Enter starting head position: ");
scanf("%d", &head);

/* Sort requests */
for(i = 0; i < r-1; i++)
{
    for(j = i+1; j < r; j++)
    {
        if(req[i] > req[j])
        {
            temp = req[i];
            req[i] = req[j];
            req[j] = temp;
        }
    }
}

printf("\nOrder of requests served:\n");

/* Move Right */
for(i = 0; i < r; i++)
{
    if(req[i] >= head)
    {

```

```

        printf("%d ", req[i]);

        total = total + abs(head - req[i]);

        head = req[i];
    }
}

/* LOOK comes back to lower requests */
for(i = r-1; i >= 0; i--)
{
    if(req[i] < head)
    {
        printf("%d ", req[i]);

        total = total + abs(head - req[i]);

        head = req[i];
    }
}

printf("\n\nTotal Head Movement = %d\n", total);

return 0;
}

```

Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames. Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

```
#include <stdio.h>
```

```

int main()
{
    int n, i, j, page, fault = 0, hit;
    int frame[10], counter[10], time = 0;
    int ref[] = {3,5,7,2,5,1,2,3,1,3,5,3,1,6,2};
    int size = 15;

```

```
printf("Enter number of frames: ");  
scanf("%d", &n);
```

```
for(i = 0; i < n; i++)  
{  
    frame[i] = -1;  
    counter[i] = 0;  
}
```

```
printf("\nPage\tFrames\n");
```

```
for(i = 0; i < size; i++)  
{  
    page = ref[i];  
    time++;  
    hit = 0;
```

```
    /* Check page in frames */
```

```
    for(j = 0; j < n; j++)  
    {  
        if(frame[j] == page)  
        {  
            hit = 1;  
            counter[j] = time;  
            break;  
        }  
    }  
}
```

```
    /* Page fault */
```

```
    if(hit == 0)  
    {
```

```

    fault++;

    /* Find empty frame */
    for(j = 0; j < n; j++)
    {
        if(frame[j] == -1)
            break;
    }

    /* If no empty frame, replace LRU page */
    if(j == n)

    {
        int lru = 0;

        for(j = 1; j < n; j++)
        {
            if(counter[j] < counter[lru])
                lru = j;
        }

        j = lru;
    }

    frame[j] = page;
    counter[j] = time;
}

printf("%d\t", page);

for(j = 0; j < n; j++)
{

```

```

        if(frame[j] == -1)
            printf("- ");
        else
            printf("%d ", frame[j]);
    }

    if(hit == 0)
        printf(" PF");

    printf("\n");
}

printf("\nTotal Page Faults = %d\n", fault);

return 0;
}

```

Slip 21

Write the simulation program for demand paging and show the page scheduling and total number of page faults according the MFU page replacement algorithm. Assume the memory of n frames. Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

```
#include <stdio.h>
```

```

int main()
{
    int n, i, j, page, fault = 0, hit;
    int frame[10], count[10];
    int ref[] = {8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
    int size = 15;

    printf("Enter number of frames: ");
    scanf("%d", &n);
}

```

```
for(i = 0; i < n; i++)
{
    frame[i] = -1;
    count[i] = 0;
}

printf("\nPage\tFrames\n");
```

```
for(i = 0; i < size; i++)
{
    page = ref[i];
    hit = 0;

    /* Check page */
    for(j = 0; j < n; j++)
    {
        if(frame[j] == page)
        {
            hit = 1;
            count[j]++;
            break;
        }
    }
}
```

```
/* Page fault */
if(hit == 0)
{
    fault++;
}
```

```
/* Find empty frame */
for(j = 0; j < n; j++)
{
```

```

        if(frame[j] == -1)
            break;
    }

    /* If no empty frame, replace MFU page */
    if(j == n)
    {
        int mfu = 0;

        for(j = 1; j < n; j++)
        {
            if(count[j] > count[mfu])
                mfu = j;
        }

        j = mfu;
    }

    frame[j] = page;
    count[j] = 1;
}

printf("%d\t", page);

for(j = 0; j < n; j++)
{
    if(frame[j] == -1)
        printf("- ");
    else
        printf("%d ", frame[j]);
}

```

```

        if(hit == 0)
            printf(" PF");

        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", fault);

    return 0;
}

```

Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int pid;

    pid = fork();

    if(pid < 0)
    {
        printf("Fork failed");
    }
    else if(pid == 0)
    {
        printf("Child process started\n");
        printf("Child PID = %d\n", getpid());
        printf("Parent PID = %d\n", getppid());

        sleep(5);
    }
}

```

```

        printf("\nAfter parent terminates:\n");
        printf("Child PID = %d\n", getpid());
        printf("New Parent PID = %d\n", getppid());
    }
    else
    {
        printf("Parent process\n");
        printf("Parent PID = %d\n", getpid());

        sleep(2);

        printf("Parent terminated\n");
        exit(0);
    }

    return 0;
}

```

Slip 22

Write a program to simulate Linked file allocation method. Assume disk with 'n' number of blocks. Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driven program with menu options as mentioned below and implement each option. a. Show Bit Vector b. Create New File c. Show Directory d. Delete File e. Exit

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int n, i, j, choice, file, blocks;
```

```
    int bit[100], start[100], size[100];
```

```
    int used[100][100];
```

```
    int fileCount = 0;
```

```
printf("Enter number of disk blocks: ");
scanf("%d", &n);
```

```
/* Randomly allocate some blocks */
for(i = 0; i < n; i++)
    bit[i] = rand() % 2;
```

```
while(1)
{
    printf("\n--- MENU ---");
    printf("\n1. Show Bit Vector");
    printf("\n2. Create New File");
    printf("\n3. Show Directory");
    printf("\n4. Delete File");
    printf("\n5. Exit");
    printf("\nEnter choice: ");
    scanf("%d", &choice);
```

```
    switch(choice)
    {
        case 1:
            printf("\nBit Vector:\n");
            for(i = 0; i < n; i++)
                printf("%d ", bit[i]);
            printf("\n");
            break;

        case 2:
            if(fileCount >= 100)
            {
                printf("Directory Full!\n");
```

```

        break;
    }

    printf("Enter number of blocks for file: ");
    scanf("%d", &blocks);

    /* Check free blocks */
    int count = 0;
    for(i = 0; i < n; i++)
    {
        if(bit[i] == 0)
            count++;
    }

    if(count < blocks)
    {
        printf("Not enough free blocks!\n");
        break;
    }

    fileCount++;
    size[fileCount - 1] = blocks;

    j = 0;
    for(i = 0; i < n && j < blocks; i++)
    {
        if(bit[i] == 0)
        {
            bit[i] = 1;
            used[fileCount - 1][j] = i;
            j++;
        }
    }

```

```
}
```

```
printf("File F%d created successfully.\n", fileCount);
```

```
printf("Blocks: ");
```

```
for(i = 0; i < blocks; i++)
```

```
    printf("%d ", used[fileCount - 1][i]);
```

```
printf("\n");
```

```
break;
```

```
case 3:
```

```
printf("\nDirectory:\n");
```

```
if(fileCount == 0)
```

```
{
```

```
    printf("No files.\n");
```

```
    break;
```

```
}
```

```
for(i = 0; i < fileCount; i++)
```

```
{
```

```
    printf("F%d : ", i + 1);
```

```
    for(j = 0; j < size[i]; j++)
```

```
        printf("%d ", used[i][j]);
```

```
    printf("\n");
```

```
}
```

```
break;
```

```
case 4:
```

```
printf("Enter file number to delete: ");  
scanf("%d", &file);
```

```
if(file < 1 || file > fileCount)  
{  
    printf("File not found!\n");  
    break;  
}
```

```
for(i = 0; i < size[file - 1]; i++)  
    bit[used[file - 1][i]] = 0;
```

```
printf("File F%d deleted successfully.\n", file);
```

```
/* Shift directory */  
for(i = file - 1; i < fileCount - 1; i++)  
{  
    size[i] = size[i + 1];  
  
    for(j = 0; j < size[i]; j++)  
        used[i][j] = used[i + 1][j];  
}
```

```
fileCount--;  
break;
```

```
case 5:  
    exit(0);
```

```
default:  
    printf("Invalid choice!\n");  
}
```

```
}
```

```
return 0;
```

}Write the program to simulate Non-preemptive Priority scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Process
```

```
{
```

```
    int pid;
```

```
    int at;
```

```
    int bt;
```

```
    int priority;
```

```
    int ct;
```

```
    int tat;
```

```
    int wt;
```

```
};
```

```
int main()
```

```
{
```

```
    int n, i, j, time = 0;
```

```
    float avgwt = 0, avgtat = 0;
```

```
    struct Process p[20], temp;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        p[i].pid = i + 1;
```

```

printf("\nEnter Arrival Time of P%d: ", i + 1);
scanf("%d", &p[i].at);

printf("Enter First CPU Burst of P%d: ", i + 1);
scanf("%d", &p[i].bt);

printf("Enter Priority of P%d: ", i + 1);
scanf("%d", &p[i].priority);
}

/* Generate next CPU burst randomly */
for(i = 0; i < n; i++)
{
    p[i].bt = p[i].bt + (rand() % 5);
}

/* Sort according to arrival time */
for(i = 0; i < n - 1; i++)
{
    for(j = i + 1; j < n; j++)
    {
        if(p[i].at > p[j].at)
        {
            temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}

printf("\nGantt Chart:\n");

```

```

for(i = 0; i < n; i++)
{
    /* CPU idle */
    if(time < p[i].at)
        time = p[i].at;

    printf("| P%d ", p[i].pid);

    time = time + p[i].bt;

    p[i].ct = time;

    p[i].tat = p[i].ct - p[i].at;

    p[i].wt = p[i].tat - p[i].bt;
}

printf("\n");

/* Display result */
printf("\nPID\tAT\tBT\tPriority\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].at,
        p[i].bt,
        p[i].priority,
        p[i].ct,
        p[i].tat,
        p[i].wt,

```

```

        p[i].wt);

    avgwt = avgwt + p[i].wt;
    avgtat = avgtat + p[i].tat;
}

avgwt = avgwt / n;
avgtat = avgtat / n;

printf("\nAverage Waiting Time = %.2f", avgwt);
printf("\nAverage Turnaround Time = %.2f\n", avgtat);

return 0;
}

```

Slip 23

2 Implement the C program to accept n integers to be sorted. Main function creates child process using fork system call. Parent process sorts the integers using bubble sort and waits for child process using wait system call child process sorts the integers using inse

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int n, a[20], i, j, temp;
    pid_t pid;

    printf("Enter number of integers: ");
    scanf("%d", &n);

    printf("Enter integers:\n");
    for(i = 0; i < n; i++)

```

```
scanf("%d", &a[i]);
```

```
pid = fork();
```

```
if(pid < 0)
```

```
{
```

```
    printf("Fork failed\n");
```

```
    return 1;
```

```
}
```

```
/* Child process - Insertion Sort */
```

```
if(pid == 0)
```

```
{
```

```
    printf("\nChild Process - Insertion Sort\n");
```

```
    for(i = 1; i < n; i++)
```

```
    {
```

```
        temp = a[i];
```

```
        j = i - 1;
```

```
        while(j >= 0 && a[j] > temp)
```

```
        {
```

```
            a[j + 1] = a[j];
```

```
            j--;
```

```
        }
```

```
        a[j + 1] = temp;
```

```
    }
```

```
    printf("Sorted array: ");
```

```
    for(i = 0; i < n; i++)
```

```
        printf("%d ", a[i]);
```

```

    printf("\n");
}

/* Parent process - Bubble Sort */
else
{
    printf("\nParent Process - Bubble Sort\n");

    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

printf("Sorted array: ");
for(i = 0; i < n; i++)
    printf("%d ", a[i]);

printf("\n");

wait(NULL);

printf("Parent waited for child process.\n");
}

```

```
return 0;
}
```

2 Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Process
```

```
{
    int pid, at, bt, ct, tat, wt;
};
```

```
int main()
```

```
{
    int n, i, j, time = 0;
    float avgwt = 0, avgtat = 0;
    struct Process p[20], temp;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++)
```

```
    {
        p[i].pid = i + 1;
```

```
        printf("\nEnter Arrival Time of P%d: ", i + 1);
```

```
        scanf("%d", &p[i].at);
```

```
        printf("Enter First CPU Burst of P%d: ", i + 1);
```

```

        scanf("%d", &p[i].bt);
    }

    /* Generate next CPU burst randomly */
    for(i = 0; i < n; i++)
    {
        p[i].bt = p[i].bt + (rand() % 5);
    }

    /* Sort according to Arrival Time */
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(p[i].at > p[j].at)
            {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

printf("\nGantt Chart:\n");

for(i = 0; i < n; i++)
{
    /* Select shortest job among arrived processes */
    for(j = i + 1; j < n; j++)
    {
        if(p[j].at <= time && p[j].bt < p[i].bt)
        {

```

```

        temp = p[i];
        p[i] = p[j];
        p[j] = temp;
    }
}

/* CPU idle */
if(time < p[i].at)
    time = p[i].at;

printf("| P%d ", p[i].pid);

time = time + p[i].bt;

p[i].ct = time;
p[i].tat = p[i].ct - p[i].at;
p[i].wt = p[i].tat - p[i].bt;
}

printf("\n");

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid,
        p[i].at,
        p[i].bt,
        p[i].ct,
        p[i].tat,
        p[i].wt);
}

```

```

        avgwt = avgwt + p[i].wt;
        avgtat = avgtat + p[i].tat;
    }

    avgwt = avgwt / n;
    avgtat = avgtat / n;

    printf("\nAverage Waiting Time = %.2f", avgwt);
    printf("\nAverage Turnaround Time = %.2f\n", avgtat);

    return 0;
}

```

Slip 24

Write the program to simulate Round Robin (RR) scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. The next CPU burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```

#include <stdio.h>

#include <stdlib.h>

int main()
{
    int n, q, i, time = 0, done;
    int at[20], bt[20], rem[20], ct[20], tat[20], wt[20];
    float awt = 0, atat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {

```

```
printf("\nEnter Arrival Time of P%d: ", i + 1);
scanf("%d", &at[i]);

printf("Enter CPU Burst of P%d: ", i + 1);
scanf("%d", &bt[i]);

bt[i] = bt[i] + (rand() % 5);
rem[i] = bt[i];
ct[i] = 0;
}
```

```
printf("\nEnter Time Quantum: ");
scanf("%d", &q);
```

```
printf("\nGantt Chart:\n");
```

```
while(1)
{
    done = 1;

    for(i = 0; i < n; i++)
    {
        if(rem[i] > 0 && at[i] <= time)
        {
            done = 0;

            printf("| P%d ", i + 1);

            if(rem[i] > q)
            {
                time = time + q;
                rem[i] = rem[i] - q;
            }
        }
    }
}
```

```

    }
    else
    {
        time = time + rem[i];
        rem[i] = 0;
        ct[i] = time;
    }
}

if(done == 1)
    break;

/* CPU idle */
if(done == 0)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        if(rem[i] > 0 && at[i] <= time)
        {
            found = 1;
            break;
        }
    }

    if(found == 0)
        time++;
}
}

```

```

printf("\n");

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i],
        tat[i], wt[i]);

    awt = awt + wt[i];
    atat = atat + tat[i];
}

printf("\nAverage Waiting Time = %.2f", awt / n);
printf("\nAverage Turnaround Time = %.2f\n", atat / n);

return 0;
}

```

2 Implement the C program that accepts an integer array. Main function forks child process. Parent process sorts an integer array and passes the sorted array to child process through the command line arguments of `execve()` system call. The child process uses `execve()` system call to load new program that uses this sorted array for performing the binary search the particular item in the array.

```

#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

#include <stdlib.h>

```

```

int main()
{

```

```
int n, a[20], i, j, temp, x;
```

```
int low, high, mid;
```

```
printf("Enter number of elements: ");
```

```
scanf("%d", &n);
```

```
printf("Enter elements:\n");
```

```
for(i = 0; i < n; i++)
```

```
    scanf("%d", &a[i]);
```

```
/* Sort array */
```

```
for(i = 0; i < n-1; i++)
```

```
{
```

```
    for(j = i+1; j < n; j++)
```

```
    {
```

```
        if(a[i] > a[j])
```

```
        {
```

```
            temp = a[i];
```

```
            a[i] = a[j];
```

```
            a[j] = temp;
```

```
        }
```

```
    }
```

```
}
```

```
printf("Sorted Array: ");
```

```
for(i = 0; i < n; i++)
```

```
    printf("%d ", a[i]);
```

```
printf("\n");
```

```
if(fork() == 0)
```

```
{
```

```

/* Child performs binary search */
printf("Enter item to search: ");
scanf("%d", &x);

low = 0;
high = n - 1;

while(low <= high)
{
    mid = (low + high) / 2;

    if(a[mid] == x)
    {
        printf("Item found at position %d\n", mid + 1);
        return 0;
    }
    else if(x < a[mid])
        high = mid - 1;
    else
        low = mid + 1;
}

printf("Item not found\n");
}
else
{
    wait(NULL);
    printf("Parent process completed.\n");
}

return 0;
}

```

Slip 25

Write a C program to illustrate the concept of orphan process. Parent process creates a child and terminates before child has finished its task. So child process becomes orphan process. (Use fork(), sleep(), getpid(), getppid())

```
#include <stdio.h>

#include <unistd.h>

int main()
{
    if(fork() == 0)
    {
        printf("Child PID = %d\n", getpid());
        printf("Parent PID = %d\n", getppid());

        sleep(5);

        printf("\nAfter parent terminates:\n");
        printf("Child PID = %d\n", getpid());
        printf("New Parent PID = %d\n", getppid());
    }
    else
    {
        printf("Parent PID = %d\n", getpid());
        printf("Parent is terminating...\n");

        sleep(1);
    }

    return 0;
}
```

Write a C program that behaves like a shell which displays the command prompt '\$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement the additional command 'search' as a. \$ search f filename

pattern : It will search the first occurrence of pattern in the given file b. \$ search a
filename pattern : It will search all the occurrence of pattern in the given file c. \$ search c
filename pattern : It will count the number of occurrence of pattern in the given file

REPEAT IN 1

Slip 26

Write the program to simulate Preemptive Priority scheduling. The arrival time and first CPU-burst and priority for different n number of processes should be input to the algorithm. The next CPU-burst should be generated

```
#include <stdio.h>

#include <stdlib.h>

int main()
{
    int n, i, time = 0, done = 0;
    int at[20], bt[20], rem[20], pr[20];
    int ct[20], tat[20], wt[20];
    int best;
    float awt = 0, atat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("\nEnter Arrival Time of P%d: ", i + 1);
        scanf("%d", &at[i]);

        printf("Enter CPU Burst of P%d: ", i + 1);
        scanf("%d", &bt[i]);

        printf("Enter Priority of P%d: ", i + 1);
        scanf("%d", &pr[i]);
```

```

    bt[i] = bt[i] + (rand() % 5);
    rem[i] = bt[i];
}

printf("\nGantt Chart:\n");

while(done < n)
{
    best = -1;

    for(i = 0; i < n; i++)
    {
        if(rem[i] > 0 && at[i] <= time)
        {
            if(best == -1 || pr[i] < pr[best])
                best = i;
        }
    }

    if(best == -1)
    {
        time++;
        continue;
    }

    printf("| P%d ", best + 1);

    rem[best]--;
    time++;

    if(rem[best] == 0)

```

```

    {
        ct[best] = time;
        done++;
    }
}

printf("\n");

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], pr[i],
        ct[i], tat[i], wt[i]);

    awt = awt + wt[i];
    atat = atat + tat[i];
}

printf("\nAverage Waiting Time = %.2f", awt / n);
printf("\nAverage Turnaround Time = %.2f\n", atat / n);

return 0;
}

```

Write the simulation program for demand paging and show the page scheduling and total number of page faults according the FIFO page replacement algorithm. Assume the memory of n frames. Reference String: 3, 4, 5, 6, 3, 4, 7, 3, 4, 5, 6, 7, 2, 4, 6

```
#include <stdio.h>
```

```

int main()
{
    int n, i, j, page;
    int frame[10];
    int fault = 0, hit;
    int pos = 0;

    int ref[] = {3,4,5,6,3,4,7,3,4,5,6,7,2,4,6};
    int size = 15;

    printf("Enter number of frames: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
        frame[i] = -1;

    printf("\nPage\tFrames\n");

    for(i = 0; i < size; i++)
    {
        page = ref[i];
        hit = 0;

        /* Check page */
        for(j = 0; j < n; j++)
        {
            if(frame[j] == page)
            {
                hit = 1;
                break;
            }
        }
    }
}

```

```

/* Page Fault */
if(hit == 0)
{
    fault++;

    frame[pos] = page;
    pos = (pos + 1) % n;
}

printf("%d\t", page);

for(j = 0; j < n; j++)
{
    if(frame[j] == -1)
        printf("- ");
    else
        printf("%d ", frame[j]);
}

if(hit == 0)
    printf(" PF");

printf("\n");
}

printf("\nTotal Page Faults = %d\n", fault);

return 0;
}

```

Slip 27

Write a simulation program for disk scheduling using SSTF algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments. 30, 10, 60, 95, 120, 150, 175 Start Head Position: 50

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int n, i, j, head, total = 0;
```

```
    int req[50], done[50];
```

```
    int min, pos, diff;
```

```
    printf("Enter total number of disk blocks: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of requests: ");
```

```
    scanf("%d", &j);
```

```
    printf("Enter request string:\n");
```

```
    for(i = 0; i < j; i++)
```

```
    {
```

```
        scanf("%d", &req[i]);
```

```
        done[i] = 0;
```

```
    }
```

```
    printf("Enter starting head position: ");
```

```
    scanf("%d", &head);
```

```
    printf("\nOrder of service: ");
```

```
    for(i = 0; i < j; i++)
```

```
    {
```

```

min = 9999;
pos = -1;

/* Find nearest request */
for(int k = 0; k < j; k++)
{
    if(done[k] == 0)
    {
        diff = abs(head - req[k]);

        if(diff < min)
        {
            min = diff;
            pos = k;
        }
    }
}

printf("%d ", req[pos]);

total = total + min;
head = req[pos];
done[pos] = 1;
}

printf("\nTotal Head Movement = %d\n", total);

return 0;
}

```

Write the simulation program to implement demand paging and show the page scheduling and total number of page faults according to the LRU (using counter method) page replacement algorithm. Assume the memory of n frames. Reference String : 3,5,7,2,5,1,2,3,1,3,5,3,1,6,2

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i, j, page, hit;
```

```
    int frame[10], count[10];
```

```
    int time = 0, fault = 0;
```

```
    int ref[] = {3,5,7,2,5,1,2,3,1,3,5,3,1,6,2};
```

```
    int size = 15;
```

```
    printf("Enter number of frames: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        frame[i] = -1;
```

```
        count[i] = 0;
```

```
    }
```

```
    printf("\nPage\tFrames\n");
```

```
    for(i = 0; i < size; i++)
```

```
    {
```

```
        page = ref[i];
```

```
        time++;
```

```
        hit = 0;
```

```
        /* Check page */
```

```
        for(j = 0; j < n; j++)
```

```
        {
```

```
            if(frame[j] == page)
```

```
{
    hit = 1;
    count[j] = time;
    break;
}
}
```

```
/* Page Fault */
```

```
if(hit == 0)
```

```
{
    fault++;
```

```
/* Find empty frame */
```

```
for(j = 0; j < n; j++)
```

```
{
    if(frame[j] == -1)
        break;
}
```

```
/* Replace LRU page */
```

```
if(j == n)
```

```
{
    int lru = 0;
```

```
for(j = 1; j < n; j++)
```

```
{
    if(count[j] < count[lru])
        lru = j;
}
```

```
j = lru;
```

```
}
```

```

        frame[j] = page;
        count[j] = time;
    }

    printf("%d\t", page);

    for(j = 0; j < n; j++)
    {
        if(frame[j] == -1)
            printf("- ");
        else
            printf("%d ", frame[j]);
    }

    if(hit == 0)
        printf(" PF");

    printf("\n");
}

printf("\nTotal Page Faults = %d\n", fault);

return 0;
}

```

Slip 28

Write a simulation program for disk scheduling using SCAN algorithm. Accept total number of disk blocks, disk request string, and current head position from the user. Display the list of requests in the order in which it is served. Also display the total number of head moments. 10, 25, 75, 90, 130, 145, 180, 55 Starting Head position= 80 Direction: Left

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

int main()
{
    int n, i, j, head, total = 0;
    int req[50], temp;

    printf("Enter total number of disk blocks: ");
    scanf("%d", &n);

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request string:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &req[i]);

    printf("Enter starting head position: ");
    scanf("%d", &head);

    /* Sort requests */
    for(i = 0; i < n-1; i++)
    {
        for(j = i+1; j < n; j++)
        {
            if(req[i] > req[j])
            {
                temp = req[i];
                req[i] = req[j];
                req[j] = temp;
            }
        }
    }
}

```

```
printf("\nOrder of service: ");

/* Move Left */
for(i = n-1; i >= 0; i--)
{
    if(req[i] < head)
    {
        printf("%d ", req[i]);
        total = total + abs(head - req[i]);
        head = req[i];
    }
}

/* Go to 0 */
total = total + head;
head = 0;

printf("0 ");

/* Move Right */
for(i = 0; i < n; i++)
{
    if(req[i] > head)
    {
        printf("%d ", req[i]);
        total = total + abs(head - req[i]);
        head = req[i];
    }
}

printf("\nTotal Head Movement = %d\n", total);
```

```
    return 0;
}
```

Write the simulation program for demand paging and show the page scheduling and total number of page faults according the MFU page replacement algorithm. Assume the memory of n frames. Reference String: 8, 5, 7, 8, 5, 7, 2, 3, 7, 3, 5, 9, 4, 6, 2

```
#include <stdio.h>
```

```
int main()
{
    int n, i, j, page, hit;
    int frame[10], count[10];
    int fault = 0;

    int ref[] = {8,5,7,8,5,7,2,3,7,3,5,9,4,6,2};
    int size = 15;

    printf("Enter number of frames: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        frame[i] = -1;
        count[i] = 0;
    }

    printf("\nPage\tFrames\n");

    for(i = 0; i < size; i++)
    {
        page = ref[i];
        hit = 0;
```

```
/* Check page */  
for(j = 0; j < n; j++)  
{  
    if(frame[j] == page)  
    {  
        hit = 1;  
        count[j]++;  
        break;  
    }  
}
```

```
/* Page Fault */  
if(hit == 0)  
{  
    fault++;
```

```
/* Find empty frame */  
for(j = 0; j < n; j++)  
{  
    if(frame[j] == -1)  
        break;  
}
```

```
/* Replace MFU page */  
if(j == n)  
{  
    int mfu = 0;  
  
    for(j = 1; j < n; j++)  
    {  
        if(count[j] > count[mfu])  
            mfu = j;
```

```

    }

    j = mfu;
}

frame[j] = page;
count[j] = 1;
}

printf("%d\t", page);

for(j = 0; j < n; j++)
{
    if(frame[j] == -1)
        printf("- ");
    else
        printf("%d ", frame[j]);
}

if(hit == 0)
    printf(" PF");

printf("\n");
}

printf("\nTotal Page Faults = %d\n", fault);

return 0;
}

```

Slip 29

Implement the C program that accepts an integer array. Main function forks child process. Parent process sorts an integer array and passes the sorted array to child process through the command line arguments of `execve()` system call. The child process uses `execve()` system call to load new program that uses this sorted array for performing the binary search the particular item in the array.

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int n, a[20], i, j, temp, x;
```

```
    int low, high, mid;
```

```
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter elements:\n");
```

```
    for(i = 0; i < n; i++)
```

```
        scanf("%d", &a[i]);
```

```
    /* Sort array */
```

```
    for(i = 0; i < n-1; i++)
```

```
    {
```

```
        for(j = i+1; j < n; j++)
```

```
        {
```

```
            if(a[i] > a[j])
```

```
            {
```

```
                temp = a[i];
```

```
                a[i] = a[j];
```

```
                a[j] = temp;
```

```
            }
```

```
        }
```

```
}
```

```
printf("Sorted Array: ");
```

```
for(i = 0; i < n; i++)
```

```
    printf("%d ", a[i]);
```

```
printf("\n");
```

```
if(fork() == 0)
```

```
{
```

```
    /* Child performs binary search */
```

```
    printf("Enter item to search: ");
```

```
    scanf("%d", &x);
```

```
    low = 0;
```

```
    high = n - 1;
```

```
    while(low <= high)
```

```
    {
```

```
        mid = (low + high) / 2;
```

```
        if(a[mid] == x)
```

```
        {
```

```
            printf("Item found at position %d\n", mid + 1);
```

```
            return 0;
```

```
        }
```

```
        else if(x < a[mid])
```

```
            high = mid - 1;
```

```
        else
```

```
            low = mid + 1;
```

```
    }
```

```

        printf("Item not found\n");
    }
    else
    {
        wait(NULL);
        printf("Parent process completed.\n");
    }

    return 0;
}

```

2 Write the program to simulate FCFS CPU-scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. The next CPU-burst should be generated randomly. The output should give Gantt chart, turnaround time and waiting time for each process. Also find the average waiting time and turnaround time.

```

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, i, j, time = 0;
    int at[20], bt[20], ct[20], tat[20], wt[20];
    int temp;
    float awt = 0, atat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("\nEnter Arrival Time of P%d: ", i + 1);
        scanf("%d", &at[i]);

        printf("Enter CPU Burst of P%d: ", i + 1);
    }
}

```

```

scanf("%d", &bt[i]);

/* Generate next CPU burst randomly */
bt[i] = bt[i] + (rand() % 5);
}

/* Sort according to Arrival Time */
for(i = 0; i < n - 1; i++)
{
    for(j = i + 1; j < n; j++)
    {
        if(at[i] > at[j])
        {
            temp = at[i];
            at[i] = at[j];
            at[j] = temp;

            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;
        }
    }
}

printf("\nGantt Chart:\n");

for(i = 0; i < n; i++)
{
    if(time < at[i])
        time = at[i];

    printf("| P%d ", i + 1);

```

```

        time = time + bt[i];

        ct[i] = time;
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }

    printf("\n");

    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            i + 1, at[i], bt[i],
            ct[i], tat[i], wt[i]);

        awt = awt + wt[i];
        atat = atat + tat[i];
    }

    printf("\nAverage Waiting Time = %.2f", awt / n);
    printf("\nAverage Turnaround Time = %.2f\n", atat / n);

    return 0;
}

```

Slip 30

Write the program to simulate Round Robin (RR) CPU-scheduling. The arrival time and first CPU-burst for different n number of processes should be input to the algorithm. Also give the time quantum as input. Assume the fixed IO waiting time (2 units). The next CPU-burst should be generated randomly. The output should give

Gantt chart, turnaround time and waiting time for
average waiting time and turnaround time

each process. Also find the

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int n, q, i, time = 0;
```

```
    int at[20], bt[20], rem[20], ct[20], tat[20], wt[20];
```

```
    float awt = 0, atat = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        printf("\nEnter Arrival Time of P%d: ", i + 1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("Enter CPU Burst of P%d: ", i + 1);
```

```
        scanf("%d", &bt[i]);
```

```
        /* Generate next CPU burst randomly */
```

```
        bt[i] = bt[i] + (rand() % 5);
```

```
        rem[i] = bt[i];
```

```
        ct[i] = 0;
```

```
    }
```

```
    printf("\nEnter Time Quantum: ");
```

```

scanf("%d", &q);

printf("\nGantt Chart:\n");

while(1)
{
    int done = 1;

    for(i = 0; i < n; i++)
    {
        if(rem[i] > 0 && at[i] <= time)
        {
            done = 0;

            printf("| P%d ", i + 1);

            if(rem[i] > q)
            {
                time = time + q;
                rem[i] = rem[i] - q;

                /* Fixed I/O waiting time */
                time = time + 2;
            }
            else
            {
                time = time + rem[i];
                rem[i] = 0;
                ct[i] = time;
            }
        }
    }
}

```

```

    if(done == 1)
        break;

/* CPU idle */
int found = 0;

for(i = 0; i < n; i++)
{
    if(rem[i] > 0 && at[i] <= time)
    {
        found = 1;
        break;
    }
}

if(found == 0)
    time++;
}

printf("\n");

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i],
        ct[i], tat[i], wt[i]);
}

```

```

        awt = awt + wt[i];
        atat = atat + tat[i];
    }

    printf("\nAverage Waiting Time = %.2f", awt / n);
    printf("\nAverage Turnaround Time = %.2f\n", atat / n);

    return 0;
}

```

Write the simulation program for demand paging and show the page scheduling and total number of page faults according the MFU page replacement algorithm. Assume the memory of n frames. Reference String: 2,5,2,8,5,4,1,2,3,2,6,1,2,5,9,8

```

#include <stdio.h>

int main()
{
    int n, i, j, page, hit;
    int frame[10], count[10];
    int fault = 0;

    int ref[] = {2,5,2,8,5,4,1,2,3,2,6,1,2,5,9,8};
    int size = 16;

    printf("Enter number of frames: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        frame[i] = -1;
        count[i] = 0;
    }
}

```

```
printf("\nPage\tFrames\n");
```

```
for(i = 0; i < size; i++)
```

```
{
```

```
    page = ref[i];
```

```
    hit = 0;
```

```
    /* Check page */
```

```
    for(j = 0; j < n; j++)
```

```
    {
```

```
        if(frame[j] == page)
```

```
        {
```

```
            hit = 1;
```

```
            count[j]++;
```

```
            break;
```

```
        }
```

```
    }
```

```
    /* Page Fault */
```

```
    if(hit == 0)
```

```
    {
```

```
        fault++;
```

```
        /* Find empty frame */
```

```
        for(j = 0; j < n; j++)
```

```
        {
```

```
            if(frame[j] == -1)
```

```
                break;
```

```
        }
```

```
        /* Replace MFU page */
```

```
        if(j == n)
```

```

{
    int mfu = 0;

    for(j = 1; j < n; j++)
    {
        if(count[j] > count[mfu])
            mfu = j;
    }

    j = mfu;
}

frame[j] = page;
count[j] = 1;
}

printf("%d\t", page);

for(j = 0; j < n; j++)
{
    if(frame[j] == -1)
        printf("- ");
    else
        printf("%d ", frame[j]);
}

if(hit == 0)
    printf(" PF");

printf("\n");
}

```

```

printf("\nTotal Page Faults = %d\n", fault);

return 0;
}

```

.1 Write a program to simulate Index file allocation method. Assume disk with 'n' number of blocks. Give value of 'n' as input. Randomly mark some block as allocated and accordingly maintain the list of free blocks Write menu driven program with menu options as mentioned above and implement each option ☐ Show Bit Vector ☐ Create New File ☐ Show Directory ☐ Exit [15 Marks]

Q.2 Write a program that demonstrates the use of nice () system call. After a child process is started using fork(), assign higher priority to the child using nice() system call.

```

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, i, j, ch, b;
    int bit[50], file[10][50];
    int f = 0, k;

    printf("Enter number of blocks: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
        bit[i] = rand() % 2;

    while(1)
    {
        printf("\n1. Show Bit Vector");
        printf("\n2. Create New File");
        printf("\n3. Show Directory");
        printf("\n4. Exit");
    }
}

```

```

printf("\nEnter choice: ");
scanf("%d", &ch);

if(ch == 1)
{
    printf("\nBit Vector: ");
    for(i = 0; i < n; i++)
        printf("%d ", bit[i]);
}

else if(ch == 2)
{
    printf("Enter number of blocks: ");
    scanf("%d", &b);

    k = 0;

    printf("File F%d created\n", f + 1);
    printf("Index Block: ");

    for(i = 0; i < n; i++)
    {
        if(bit[i] == 0)
        {
            bit[i] = 1;
            file[f][k] = i;
            k++;

            if(k == b + 1)
                break;
        }
    }
}

```

```

printf("%d\n", file[f][0]);

printf("Data Blocks: ");
for(i = 1; i < k; i++)
    printf("%d ", file[f][i]);

    f++;
}

else if(ch == 3)
{
    printf("\nDirectory:\n");

    for(i = 0; i < f; i++)
    {
        printf("F%d -> Index Block = %d : ",
            i + 1, file[i][0]);

        for(j = 1; j < 50 && file[i][j] != 0; j++)
            printf("%d ", file[i][j]);

        printf("\n");
    }
}

else if(ch == 4)
{
    exit(0);
}

else

```

```

        {
            printf("Invalid choice");
        }
    }

    return 0;
}

2-
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pid;

    pid = fork();

    if(pid == 0)
    {
        printf("Child Process\n");

        printf("Nice Value = %d\n", nice(0));

        nice(-5);

        printf("New Nice Value = %d\n", nice(0));

        printf("Child priority increased\n");
    }
    else
    {

```

```
wait(NULL);  
printf("Parent Process\n");  
}  
  
return 0;  
}
```

=====BEST OF LUCK=====